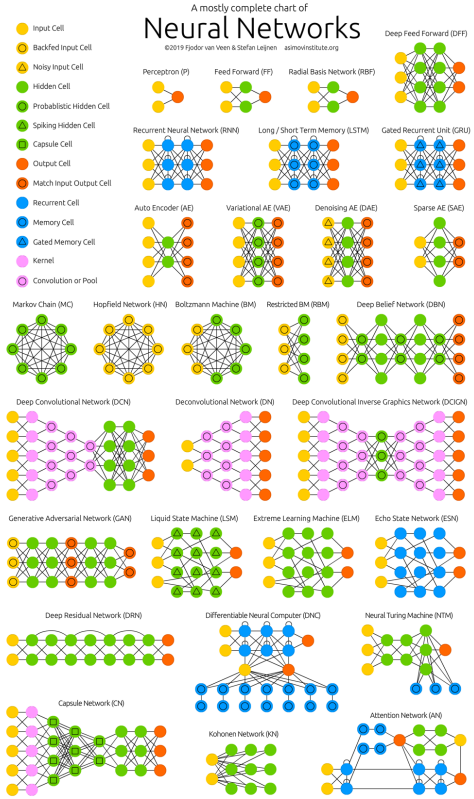
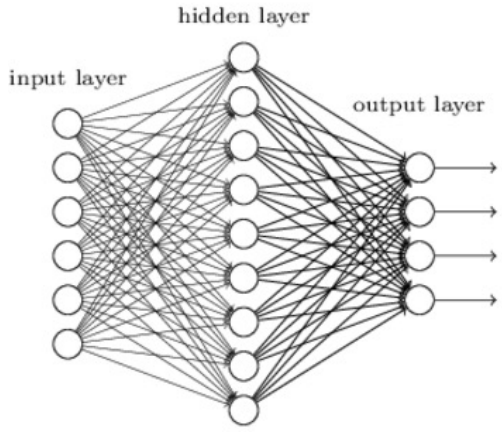
Neural Networks

Neural Nets

- The basics of ”Deep Learning”.



"Non-deep" feedforward neural network



Deep neural network

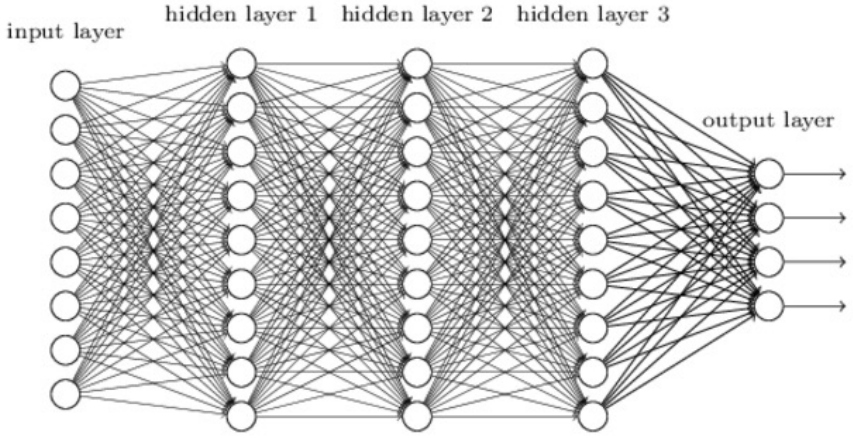
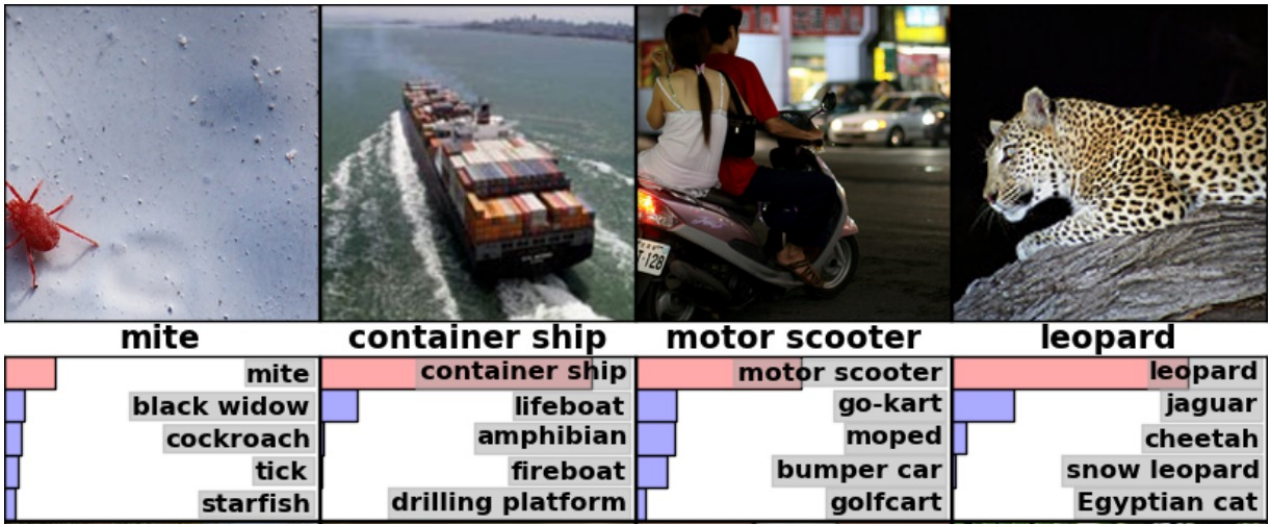


Image Recognition

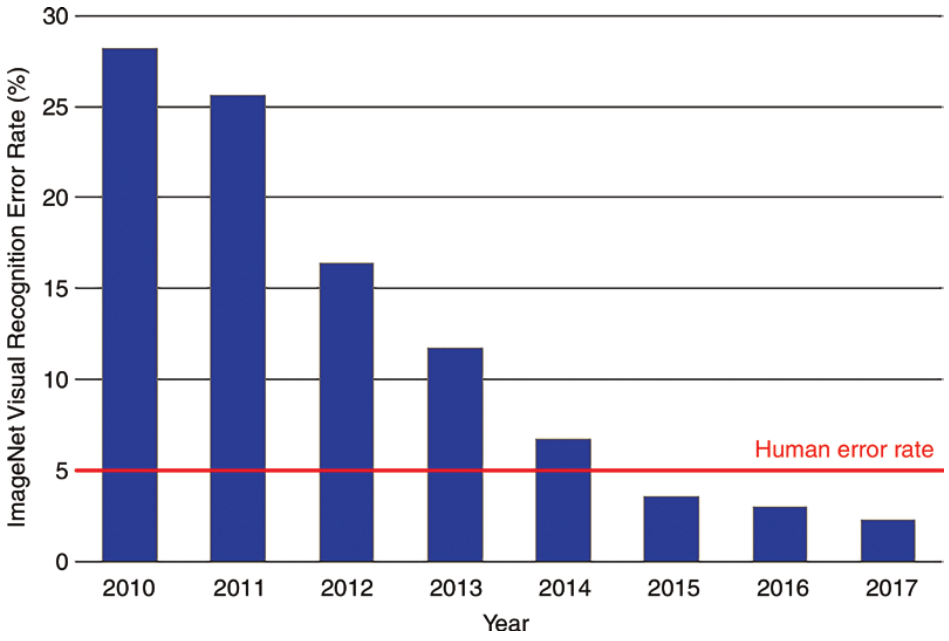
- Previously a very hard task in CS:

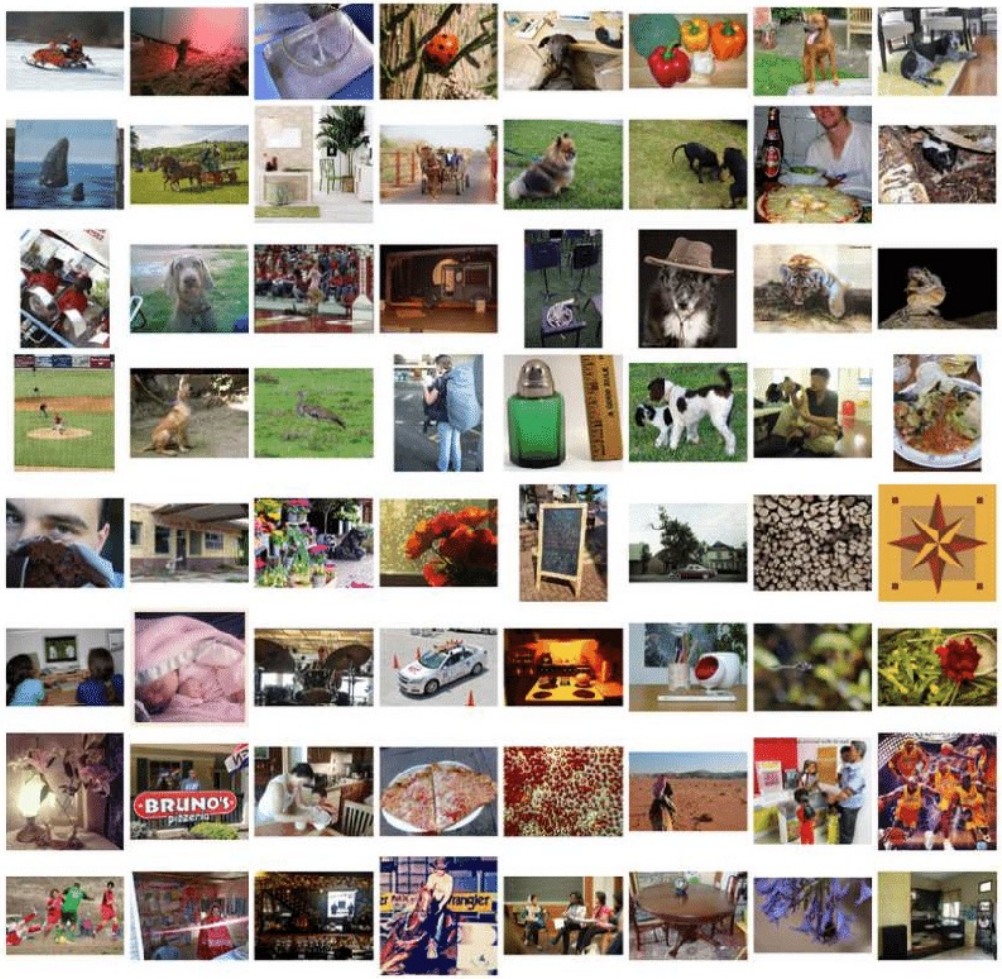




Deep Learning Breakthrough

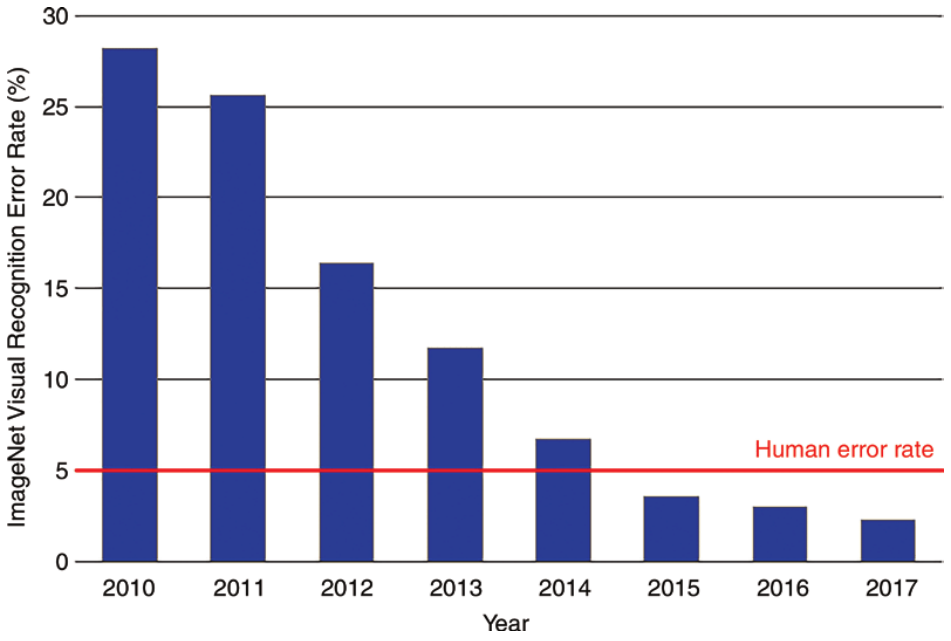
- 2012: Started using deep neural nets. AlexNet.
  - Better than humans on ImageNet data set.
  - Still not better "in the wild".

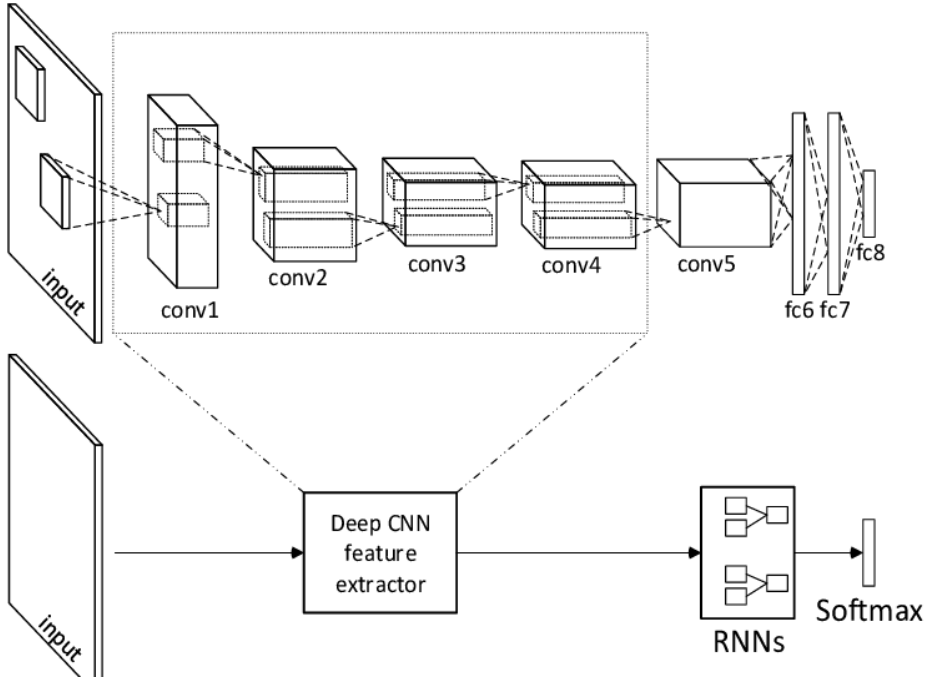




Deep Learning Breakthrough

- AlexNet (2012): 60 million parameters.
  - Trained on 1 million images.
- FixEfficientNet-L2 (2020): 480 million parameters.
  - Trained on >300 million extra images.





Deep Learning Breakthrough

- Machine learning models based on learning multiple levels of abstraction.
- Amazing improvements in object recognition, object detection, speech recognition, natural language processing and reinforcement learning.









- You need lots of data.
- Need powerful GPUs.

Deep Learning

- Machine learning models based on learning multiple levels of abstraction.

Deep Learning: Automating Feature Discovery

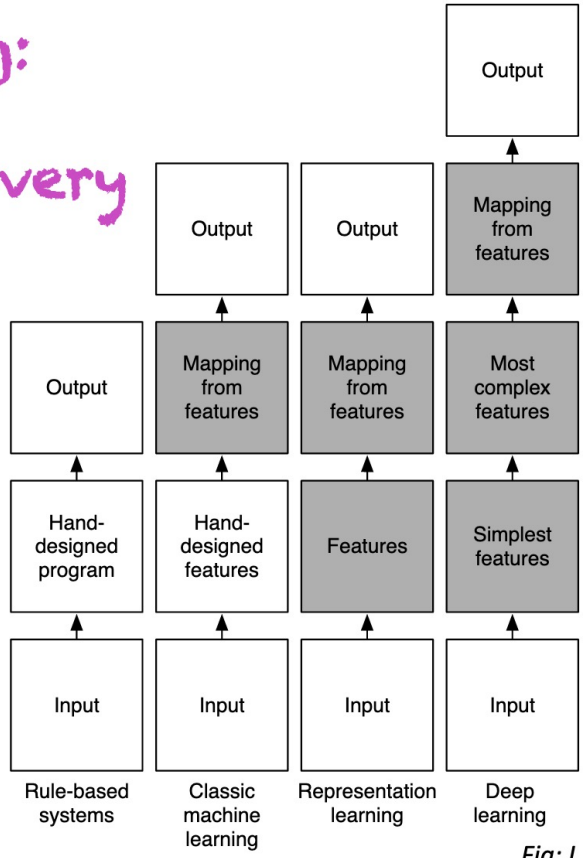




Fig: I. Goodfellow

Training

- How do we train such powerful models?

- (Mini-Batch) Stochastic Gradient Descent!
  - With fancy control of learning rate and tricks to avoid overfitting.

- Nothing more but small steps in the negative direction of the gradient (with respect to the model parameters).

- You already know most of the tools by now!

Coming Up

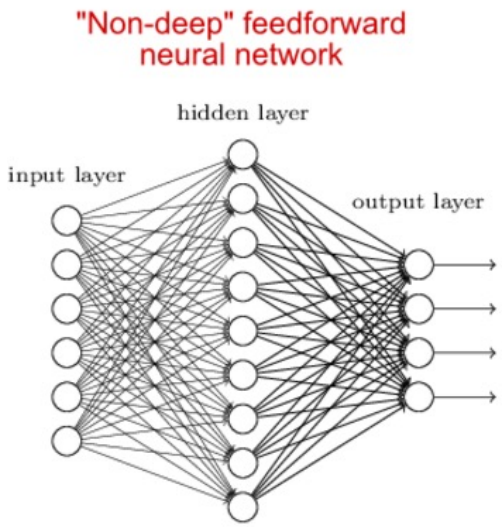
- Neural Nets – Nonlinear hypothesis set.
- Understand what a neural net is and how we can use it for supervised learning.
- Feed forward neural nets.
- Afterwards: How to train them.

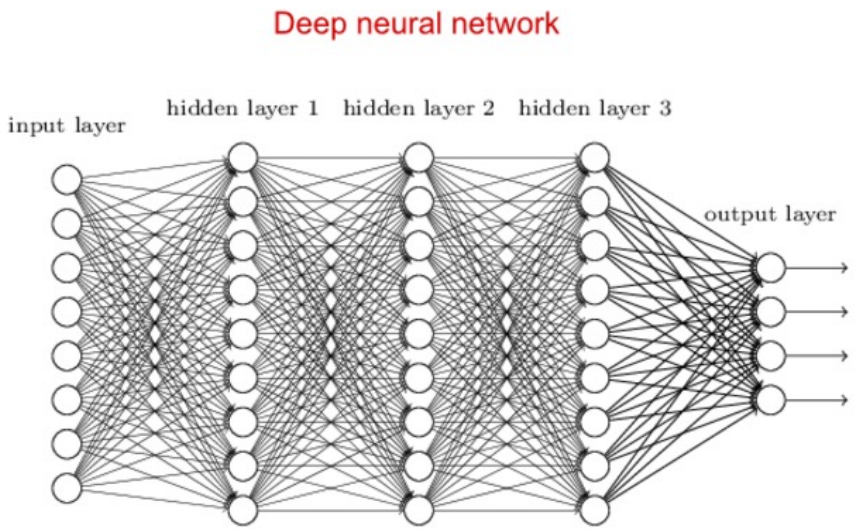


Neural Networks

Neural Nets

- The basics of ”Deep Learning”.





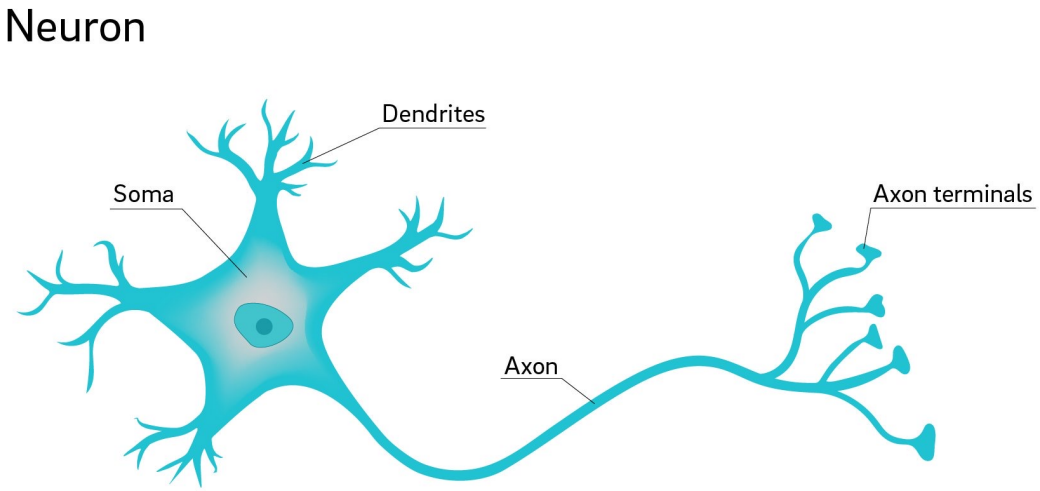
Humans

- Some think the best learner is the human brain (for many tasks)
- Humans are good at some things ”computers are not” (used not to be)
  - Recognizing images
  - Playing games
  - Understanding human language
- Want to understand human brain
  to figure out how it learns
- Inspiration for new machine learning models



Human Brain

- Neuron:
  - Tree that collects inputs from other neurons.



  - Output edges that connect to other neurons' inputs.
  - If "enough" input is given, neuron fires to other neurons.
  - Neurons change over life (learn).

Human Brain

- Overly Simplified Abstraction:
  - Each neuron receives inputs from other neurons.
  - The effect of each input line on the neuron is controlled by a synaptic weight
    - The weights can be positive or negative (and zero).
    - If ”good” input -> fire input to other neurons.
  - The synaptic weights adapt so the whole network learns to perform useful computations.
    - Recognizing objects, understanding language, making plans, moving the body.
  - Brain has 10^{11} neurons each with about 10^4 weights.
  - A huge number of weights affect computation in a short amount of time (massively parallel).

Neural Nets

Brain Inspiration

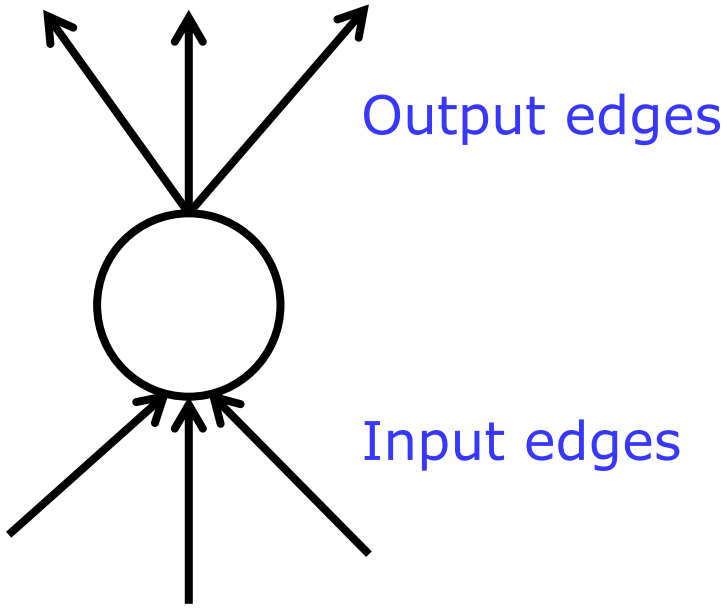


Mathematical Models
for Machine Learning

NOT a model of the brain!

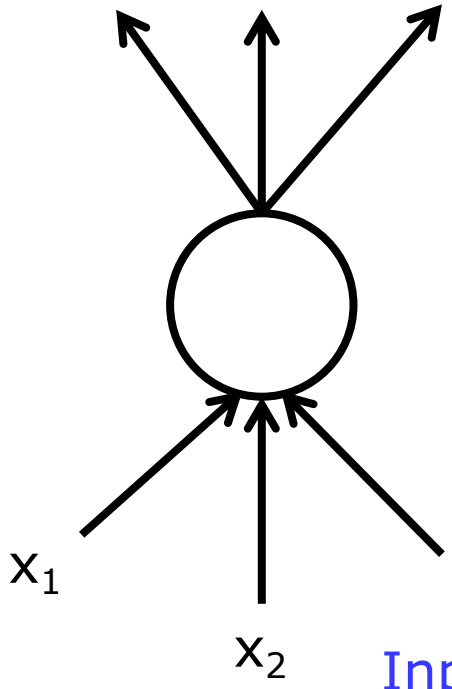
Neural Nets

- Neurons:
  - The basic building block.



Neural Nets

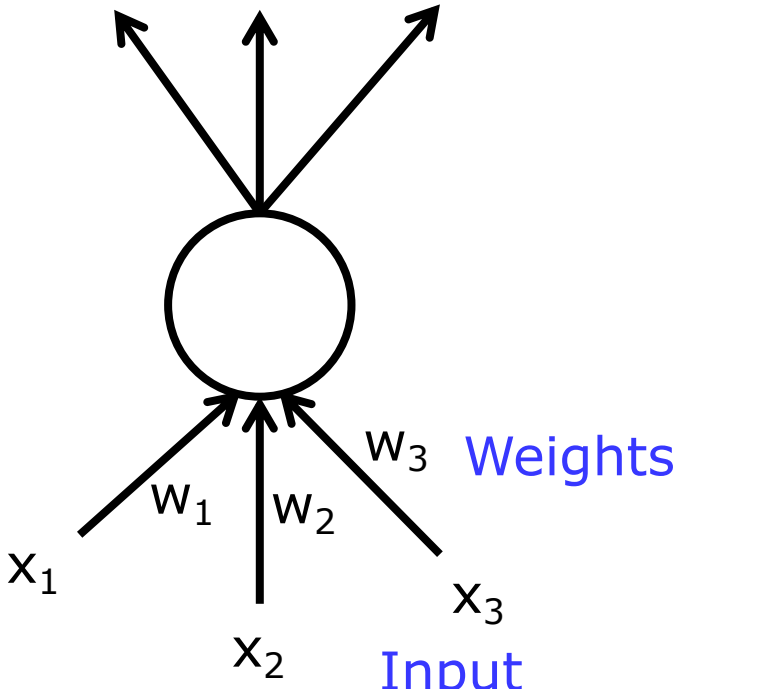
- Neurons:
  - The basic building block.



Input values/signals

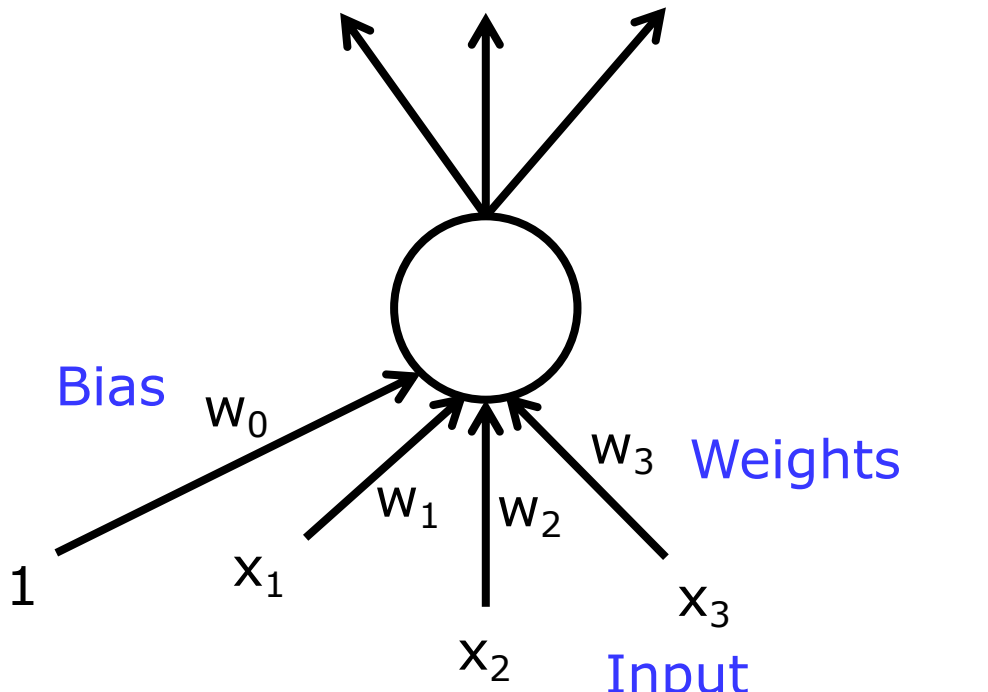
Neural Nets

- Neurons:
  - The basic building block.



Neural Nets

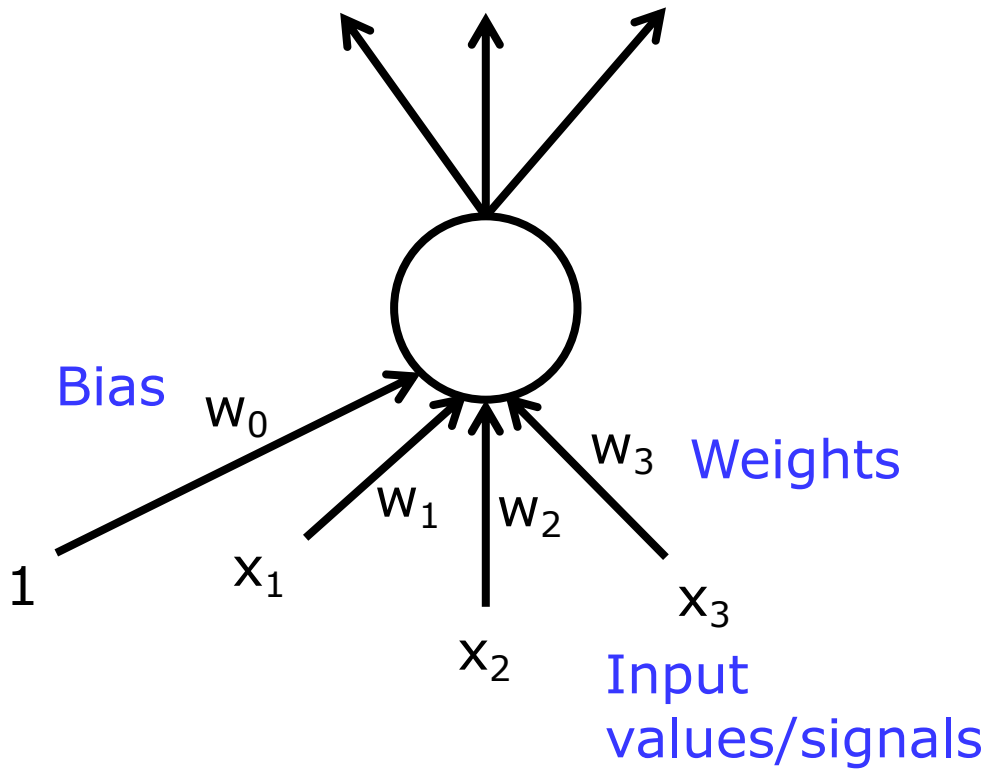
- Neurons:
  - The basic building block.



Like linear models

Neural Nets

- Neurons:
  - The basic building block.

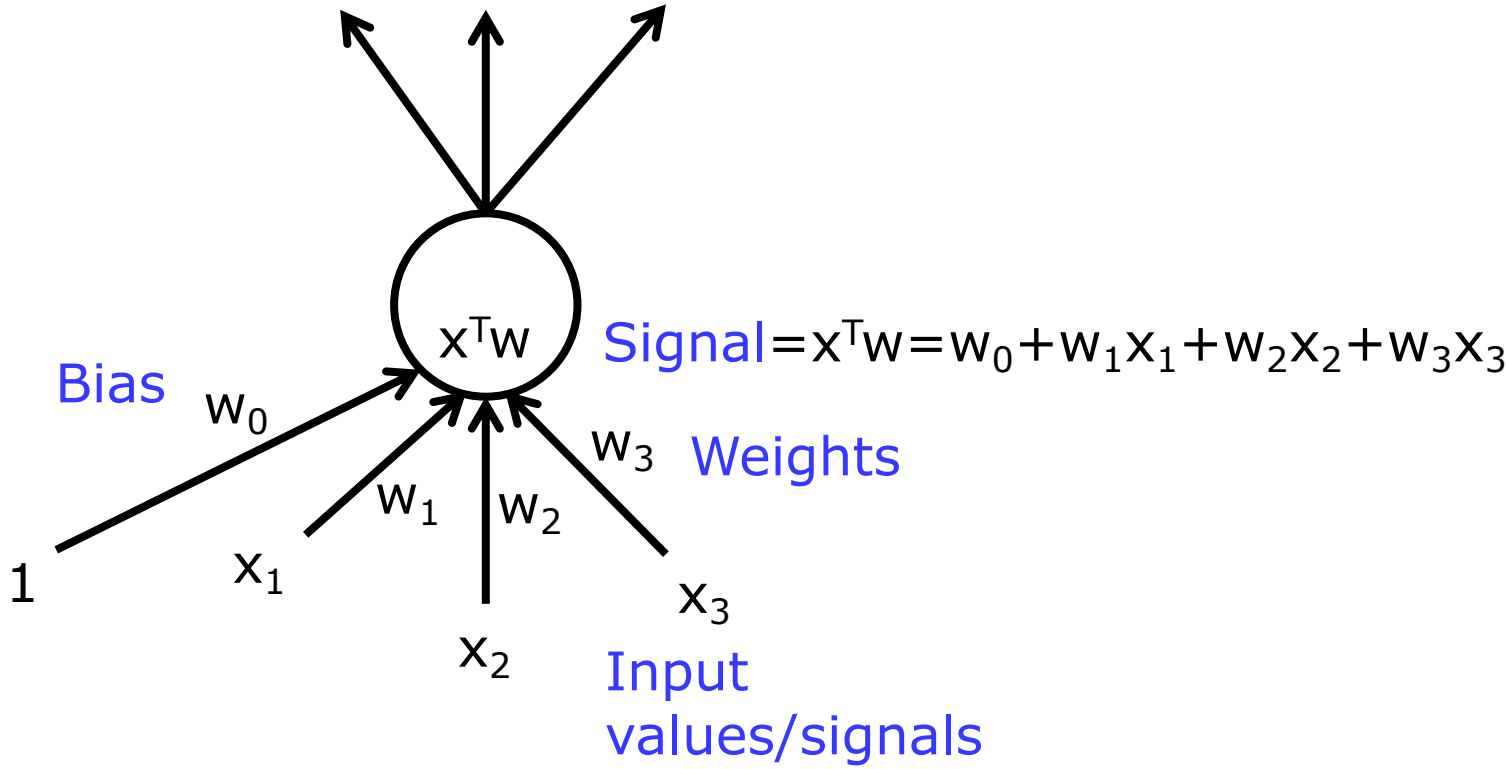


Neural Nets

$x=(1,x_1,x_2,x_3)^T$

$w=(w_0,w_1,w_2,w_3)^T$

- Neurons:
  - The basic building block.

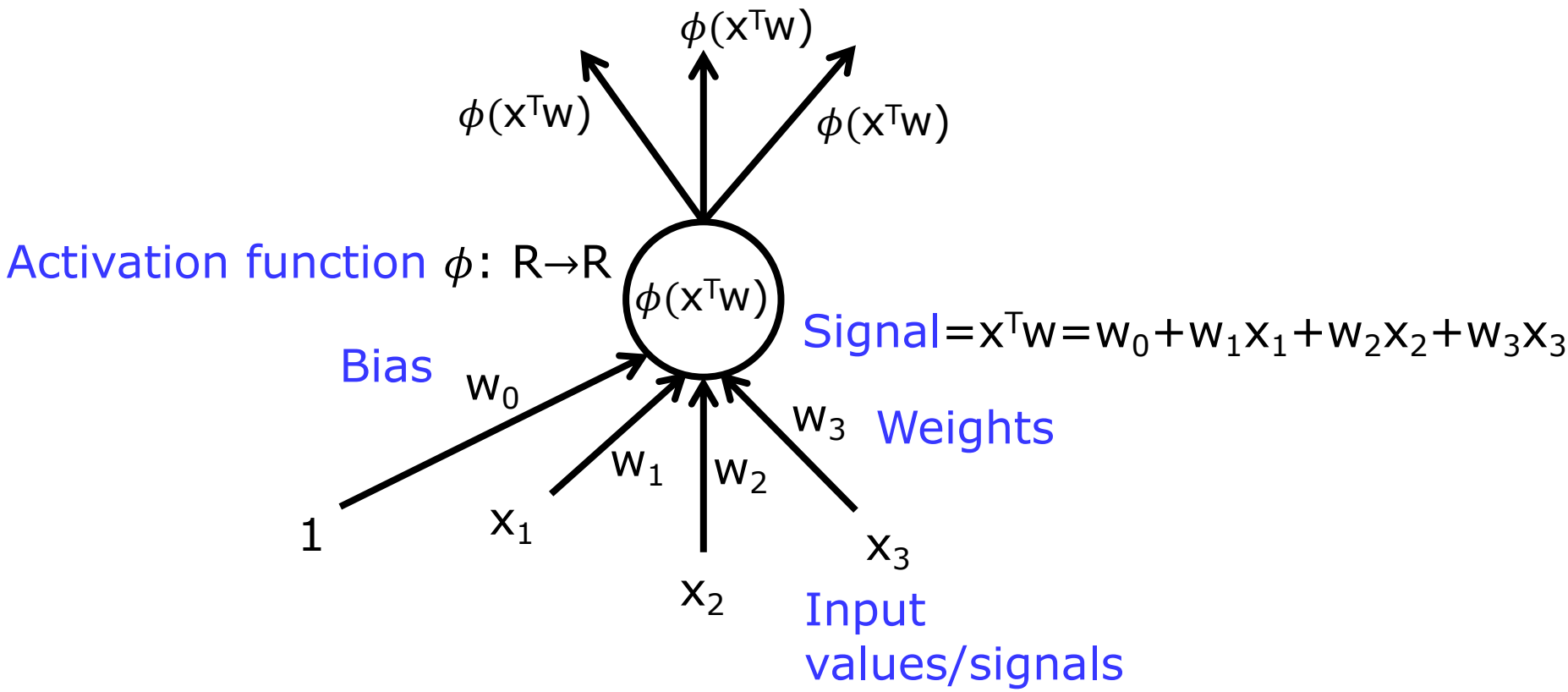


Neural Nets

$x=(1,x_1,x_2,x_3)^T$

$w=(w_0,w_1,w_2,w_3)^T$

- Neurons:
  - The basic building block.



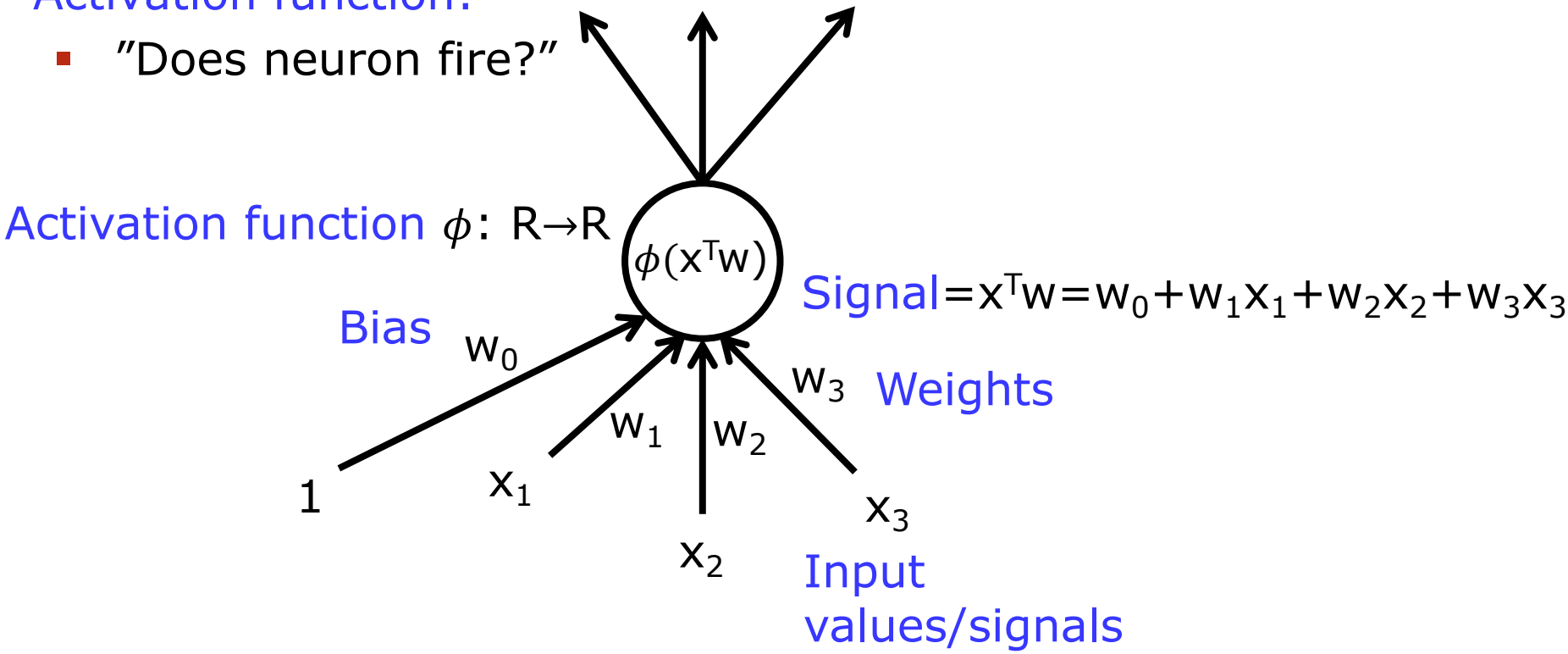
Neural Nets

$x=(1,x_1,x_2,x_3)^T$

$w=(w_0,w_1,w_2,w_3)^T$

- Neurons:
  - The basic building block.

- Activation function:
  - ”Does neuron fire?”



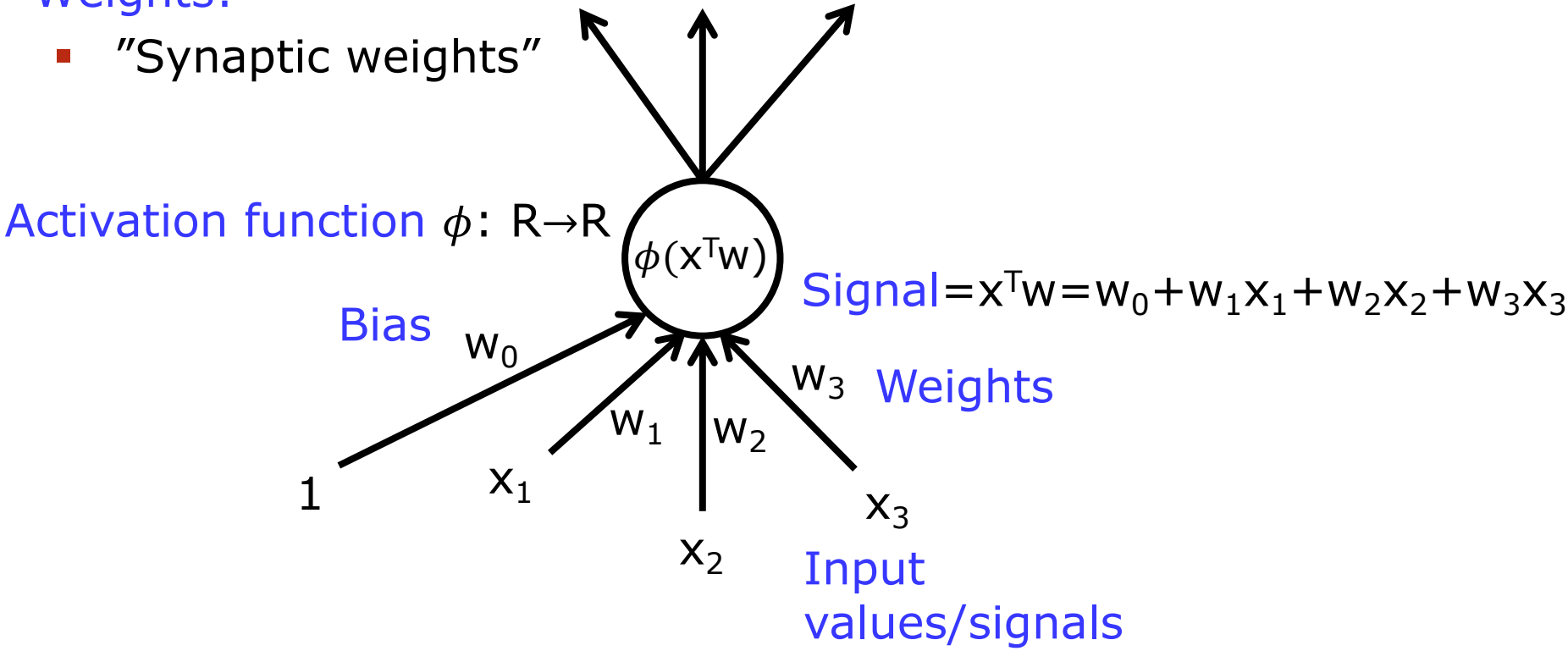
Neural Nets

$x=(1,x_1,x_2,x_3)^T$

$w=(w_0,w_1,w_2,w_3)^T$

- Neurons:
  - The basic building block.

- Weights:
  - ”Synaptic weights”



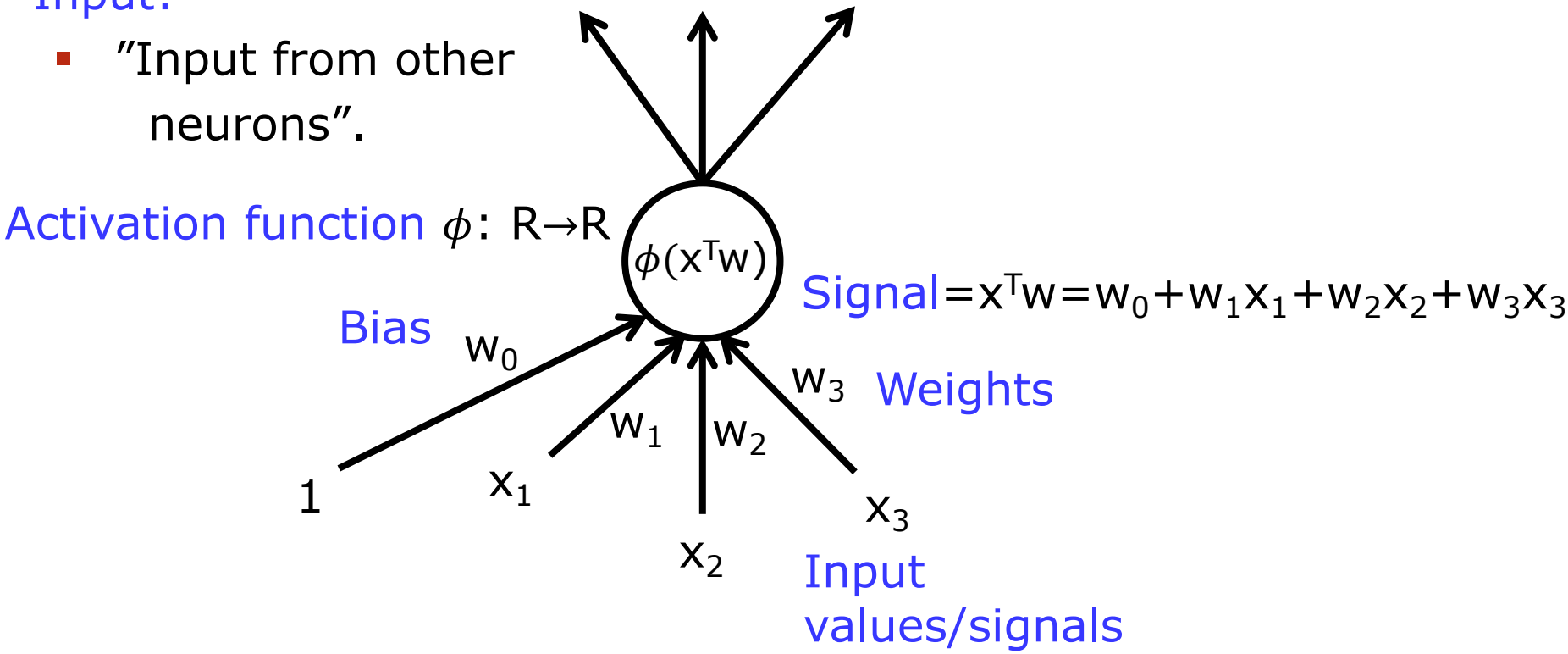
Neural Nets

$x=(1,x_1,x_2,x_3)^T$

$w=(w_0,w_1,w_2,w_3)^T$

- Neurons:
  - The basic building block.

- Input:
  - ”Input from other neurons”.



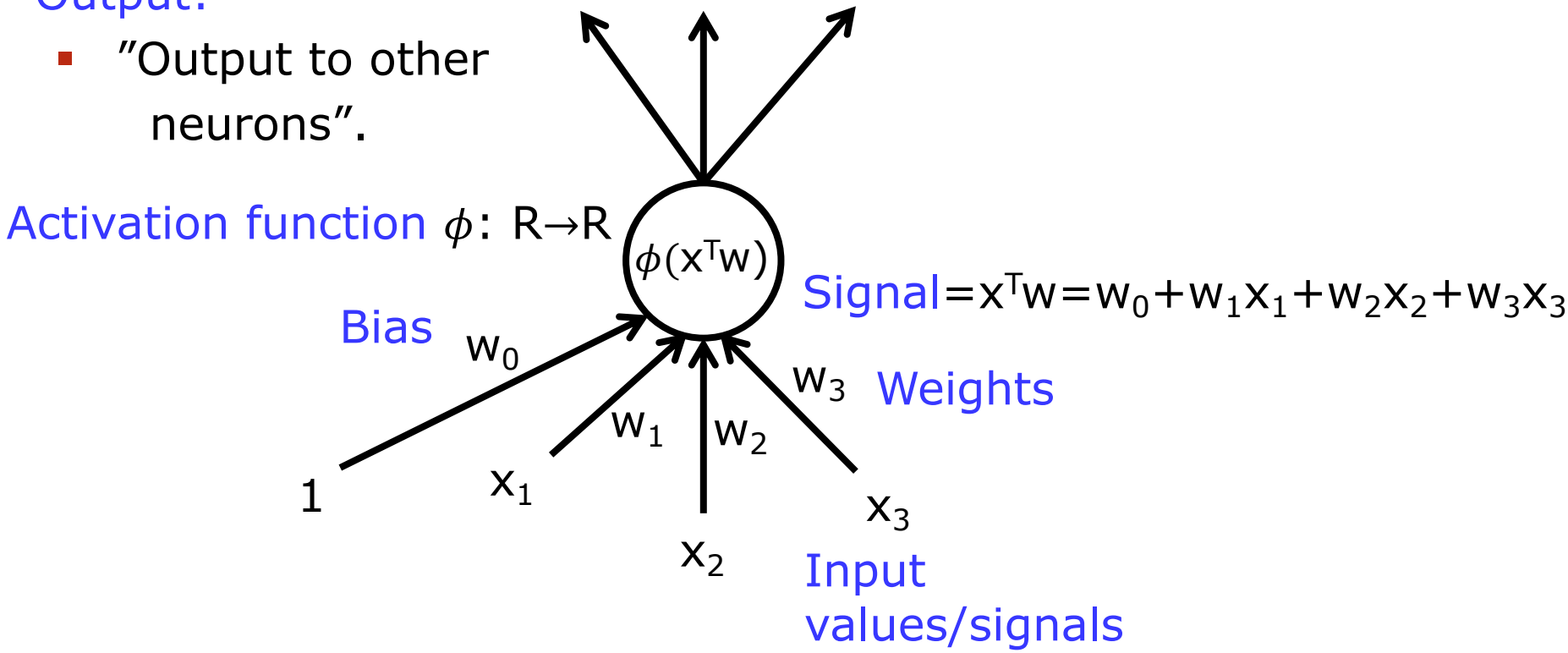
Neural Nets

$x=(1,x_1,x_2,x_3)^T$

$w=(w_0,w_1,w_2,w_3)^T$

- Neurons:
  - The basic building block.

- Output:
  - ”Output to other neurons”.

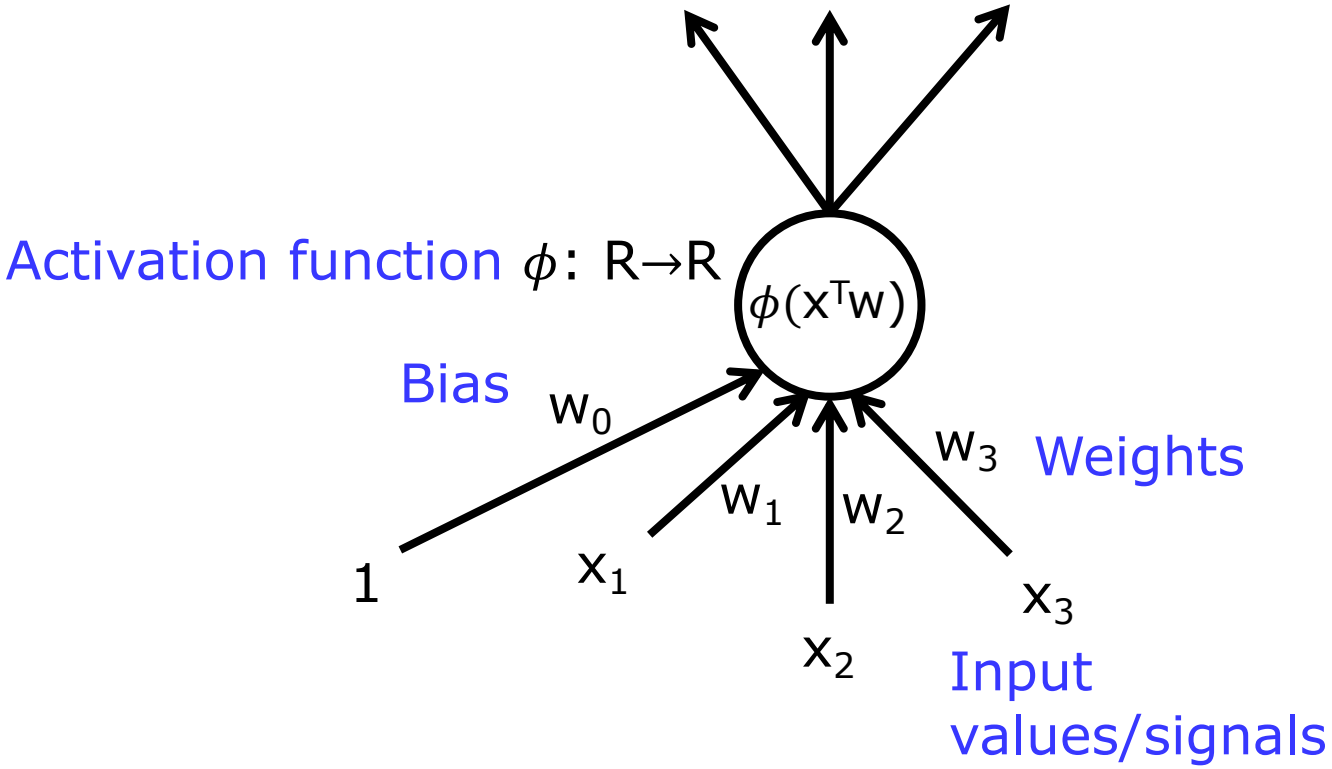


Neural Nets

$x=(1,x_1,x_2,x_3)^T$

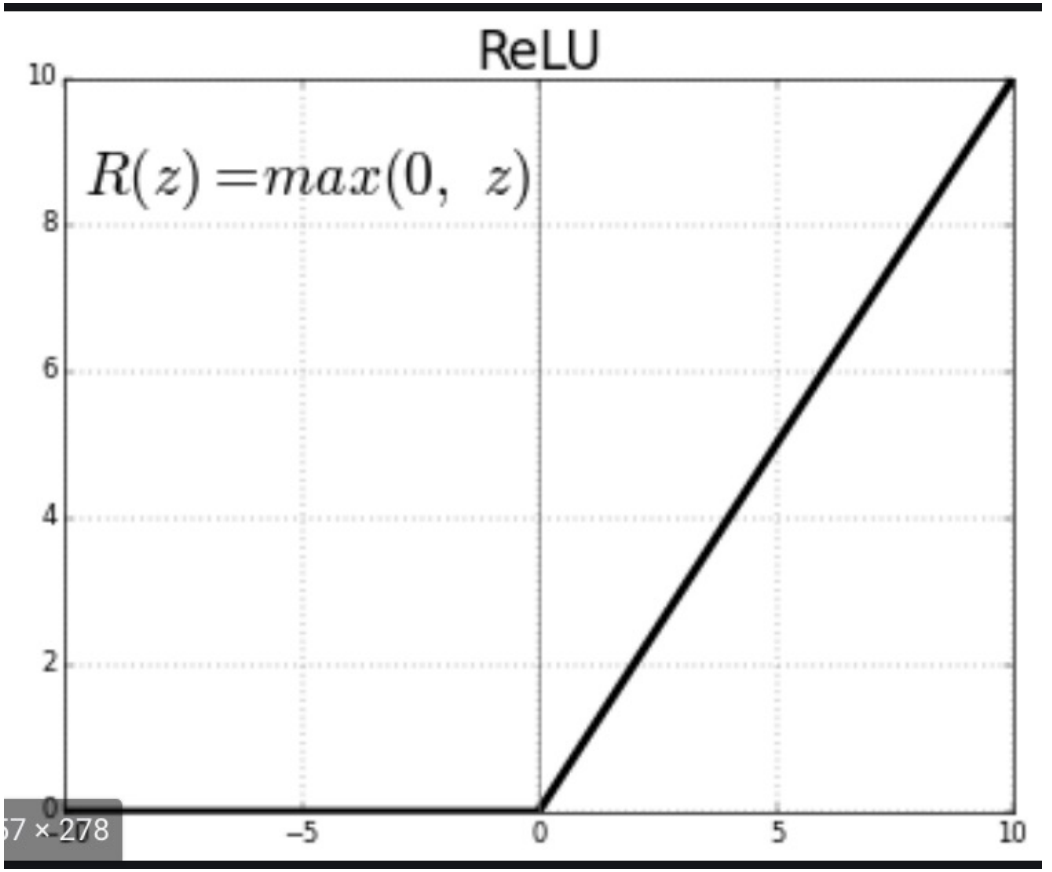
$w=(w_0,w_1,w_2,w_3)^T$

- Neurons:
  - The basic building block.



Neural Nets

- Activation function ϕ: R→R
  - ”Should neuron fire?”
- Very popular ϕ
  - ReLU: Rectified Linear Unit.
- ReLU $\phi(z)$
  - Linear for z>0.
  - Constant for z≤0.
  - Non-linear function.
- ”Fires” for z>0.

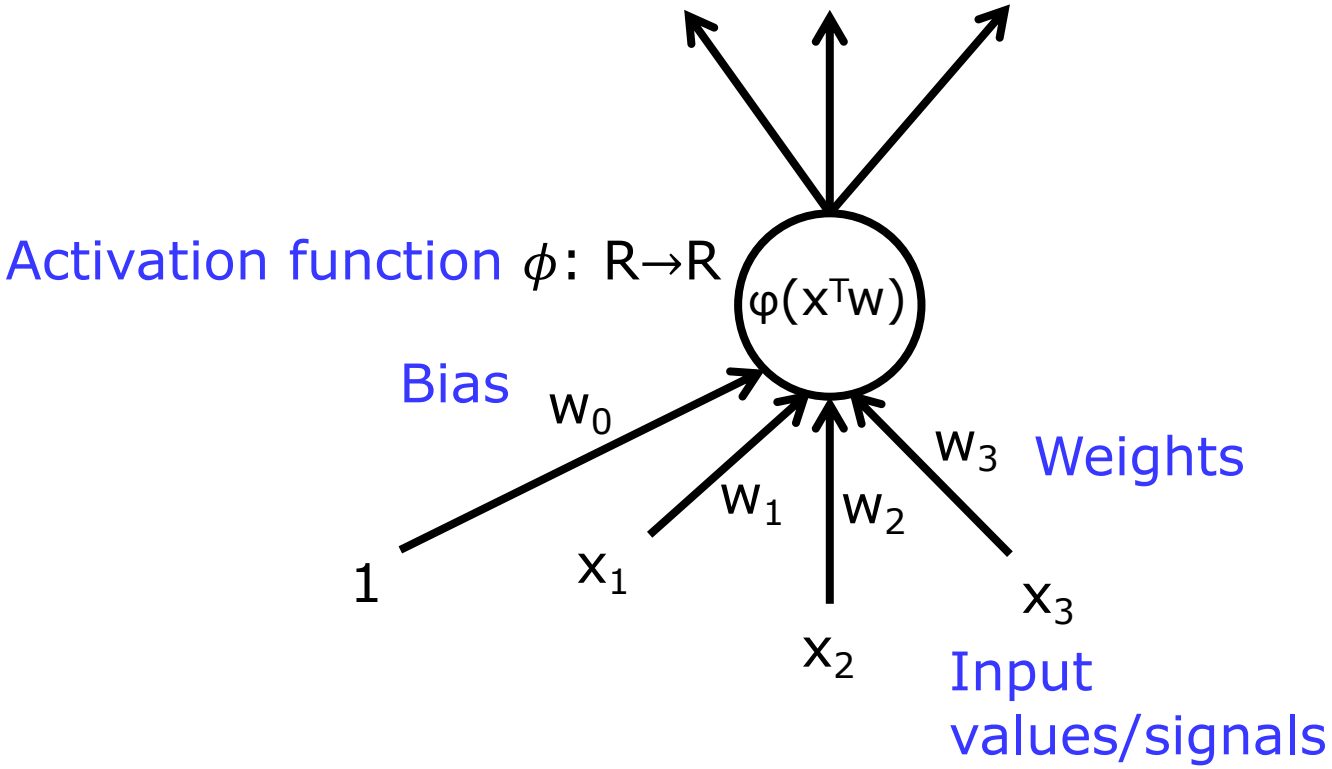


Neural Nets

$x=(1,x_1,x_2,x_3)^T$

$w=(w_0,w_1,w_2,w_3)^T$

- Neurons:
  - The basic building block.

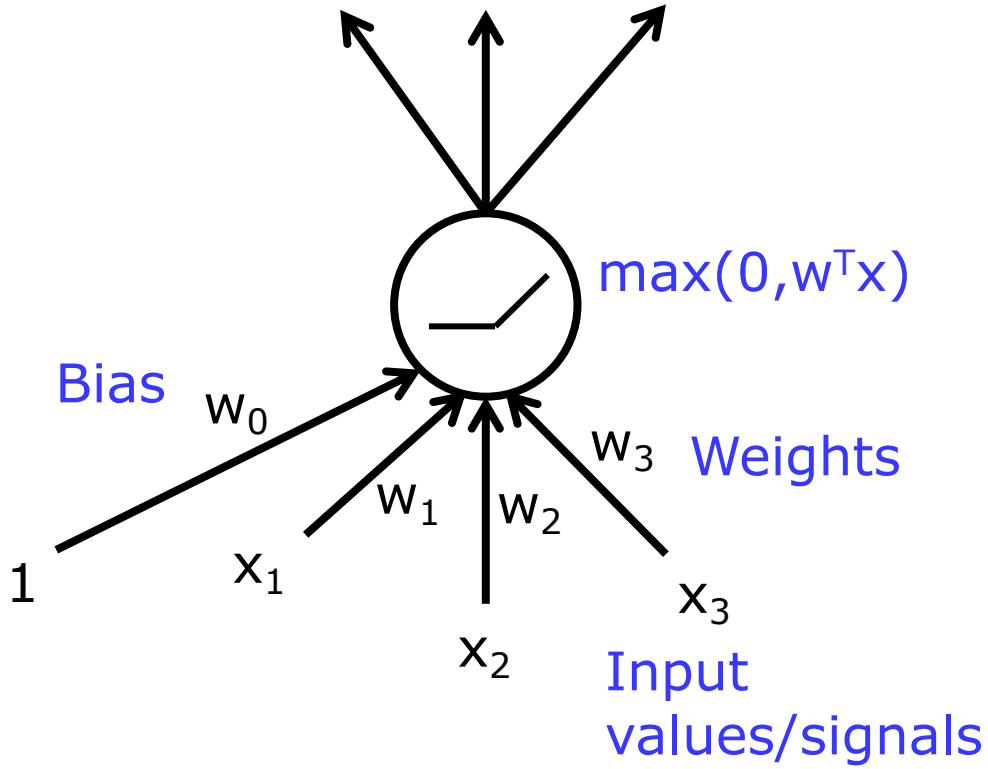


Neural Nets

$x=(1,x_1,x_2,x_3)^T$

$w=(w_0,w_1,w_2,w_3)^T$

- Neurons:
  - The basic building block.

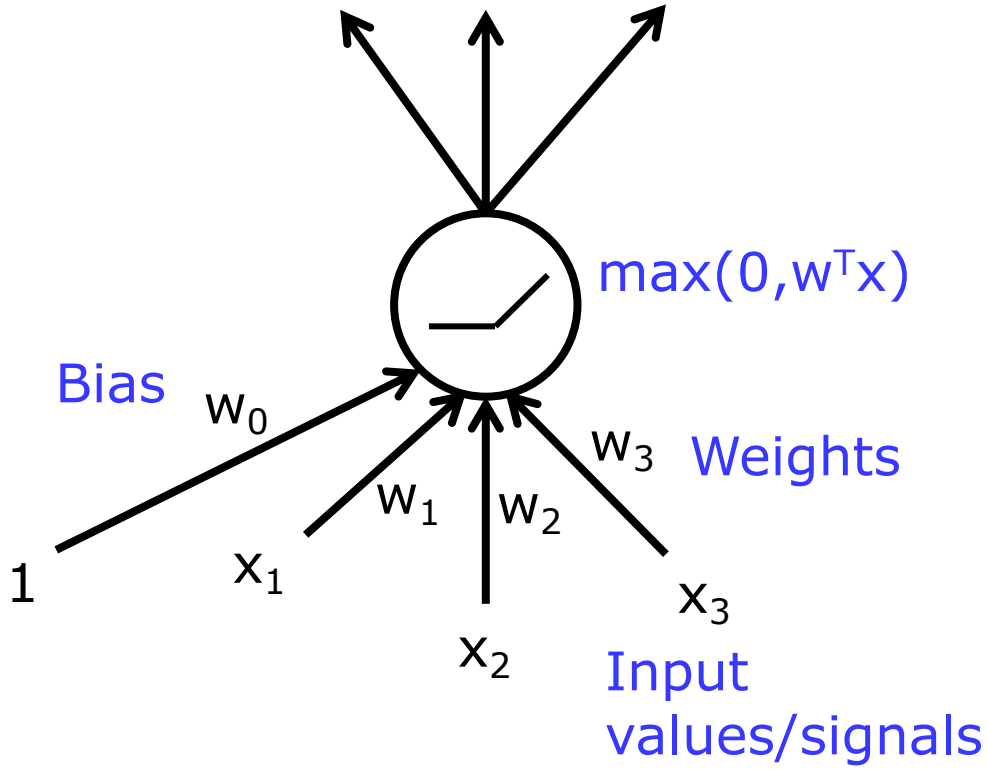


Neural Nets

$x=(1,x_1,x_2,x_3)^T$

$w=(w_0,w_1,w_2,w_3)^T$

- Neurons:
  - The basic building block.
  - Weights w are the parameters to train.

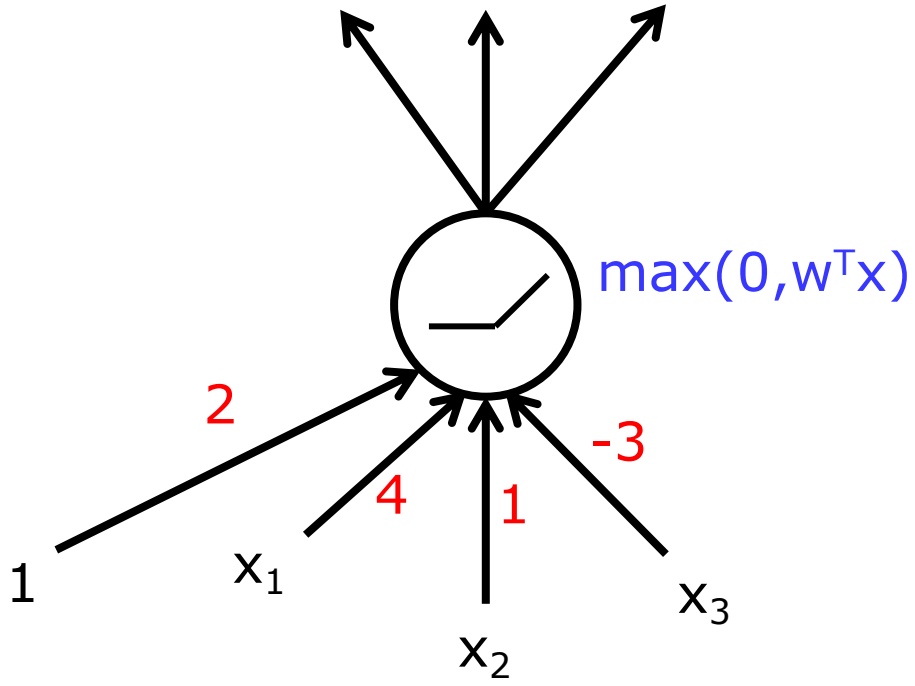


Neural Nets

$x=(1,x_1,x_2,x_3)^T$

$w=(2,4,1,-3)^T$

- Example Evaluation

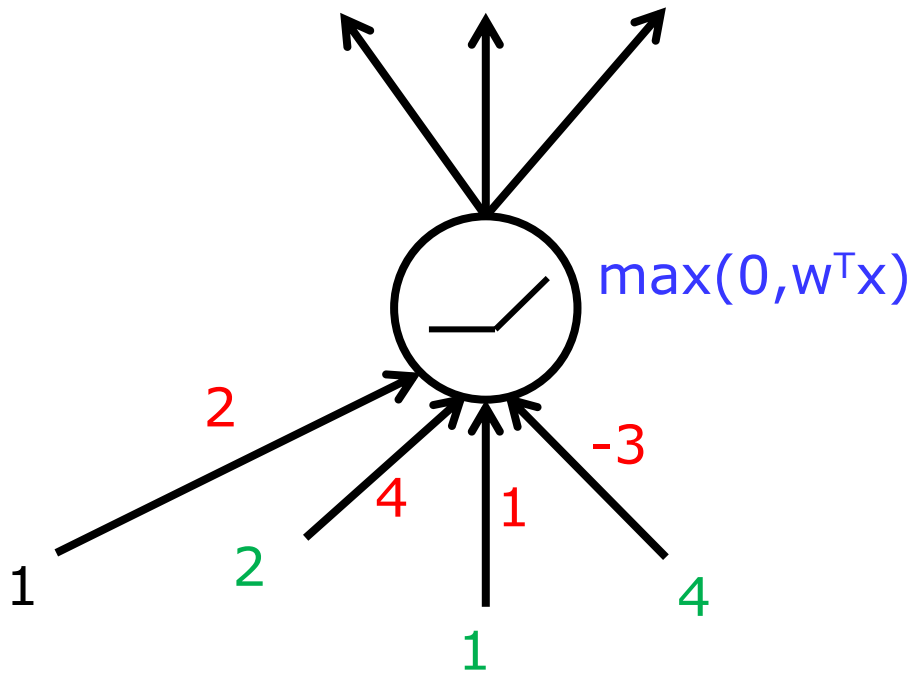


Neural Nets

$x=(1,2,1,4)^T$

$w=(2,4,1,-3)^T$

- Example Evaluation

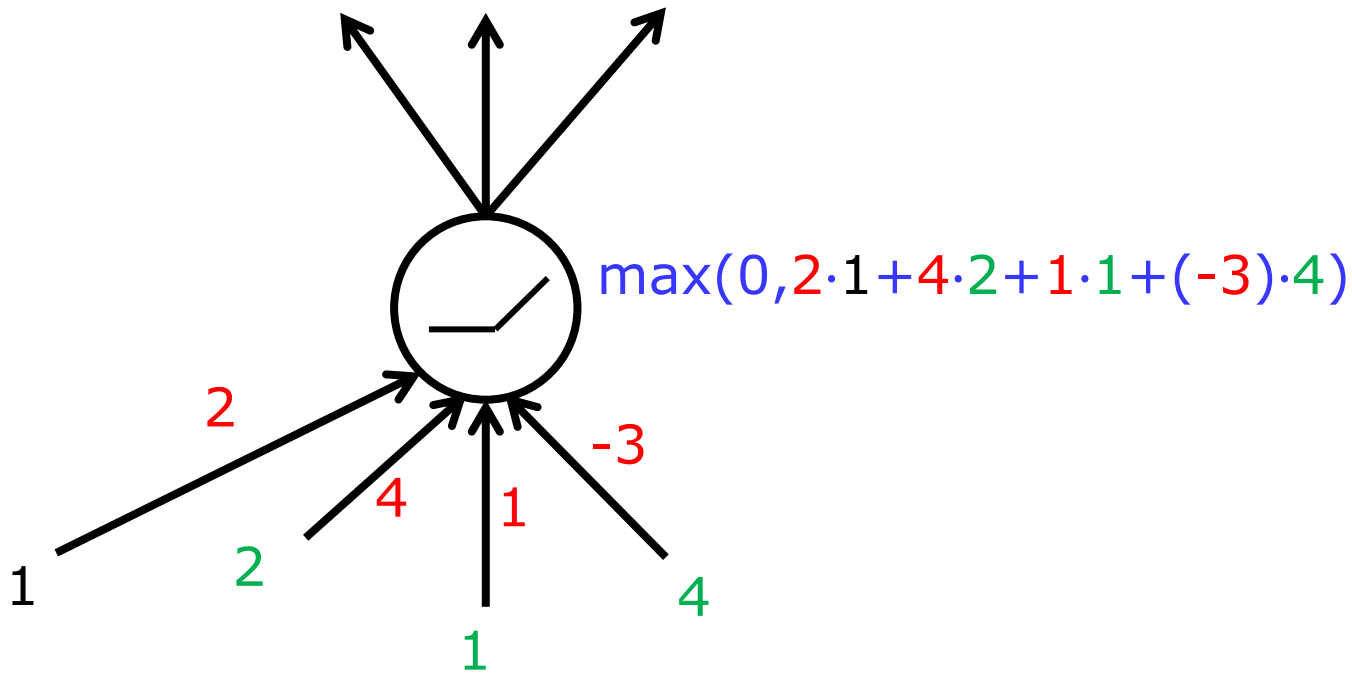


Neural Nets

- Example Evaluation

$x=(1,2,1,4)^T$

$w=(2,4,1,-3)^T$

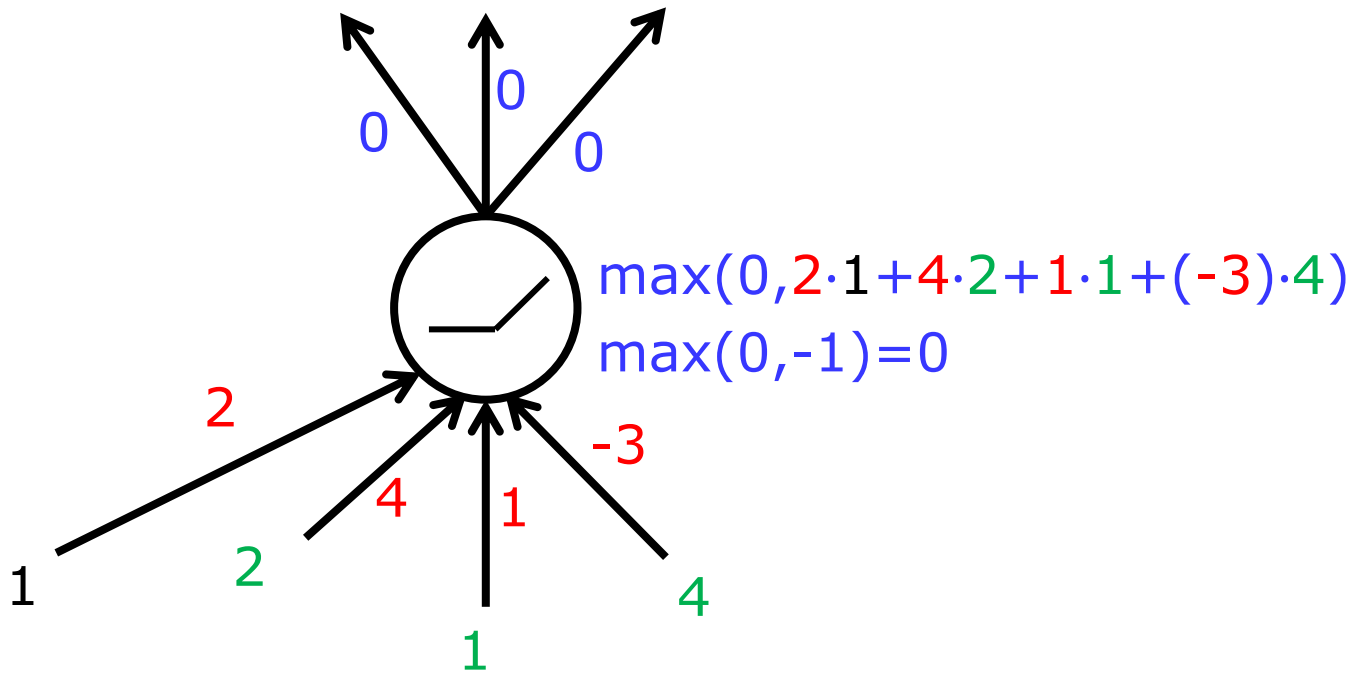


Neural Nets

- Example Evaluation

$x=(1,2,1,4)^T$

$w=(2,4,1,-3)^T$

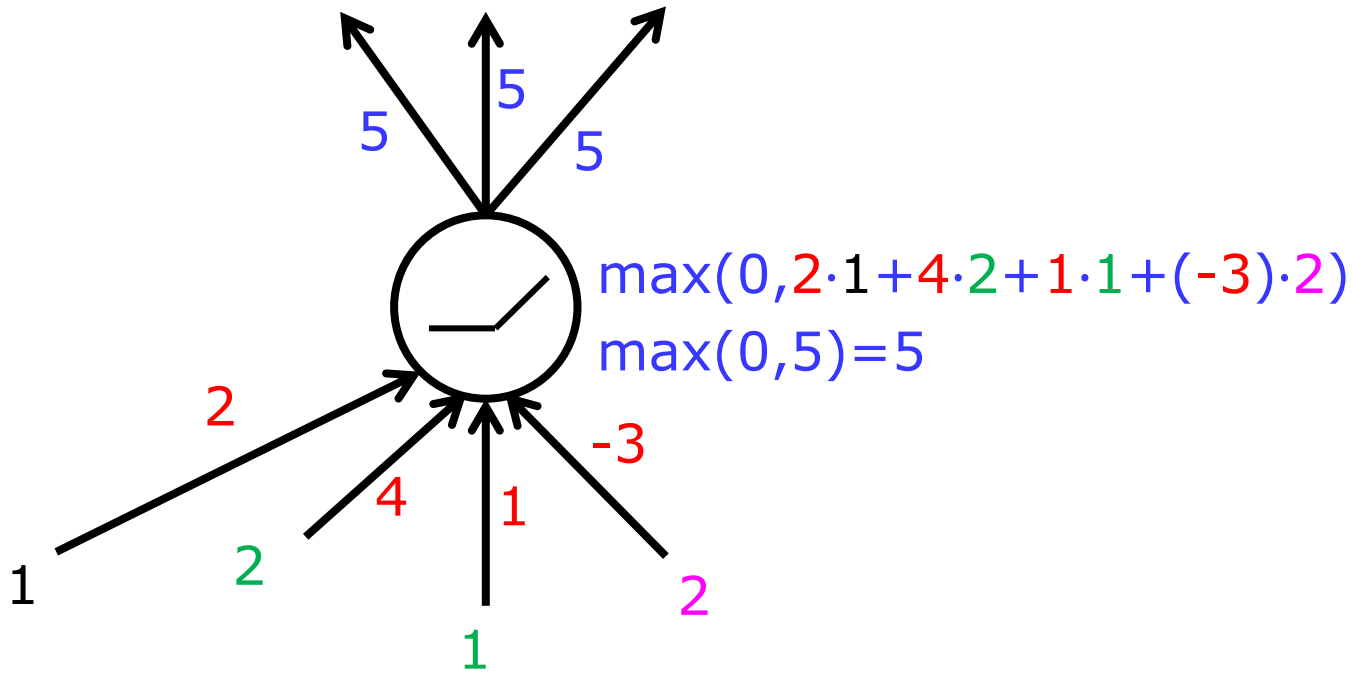


Neural Nets

- Example Evaluation

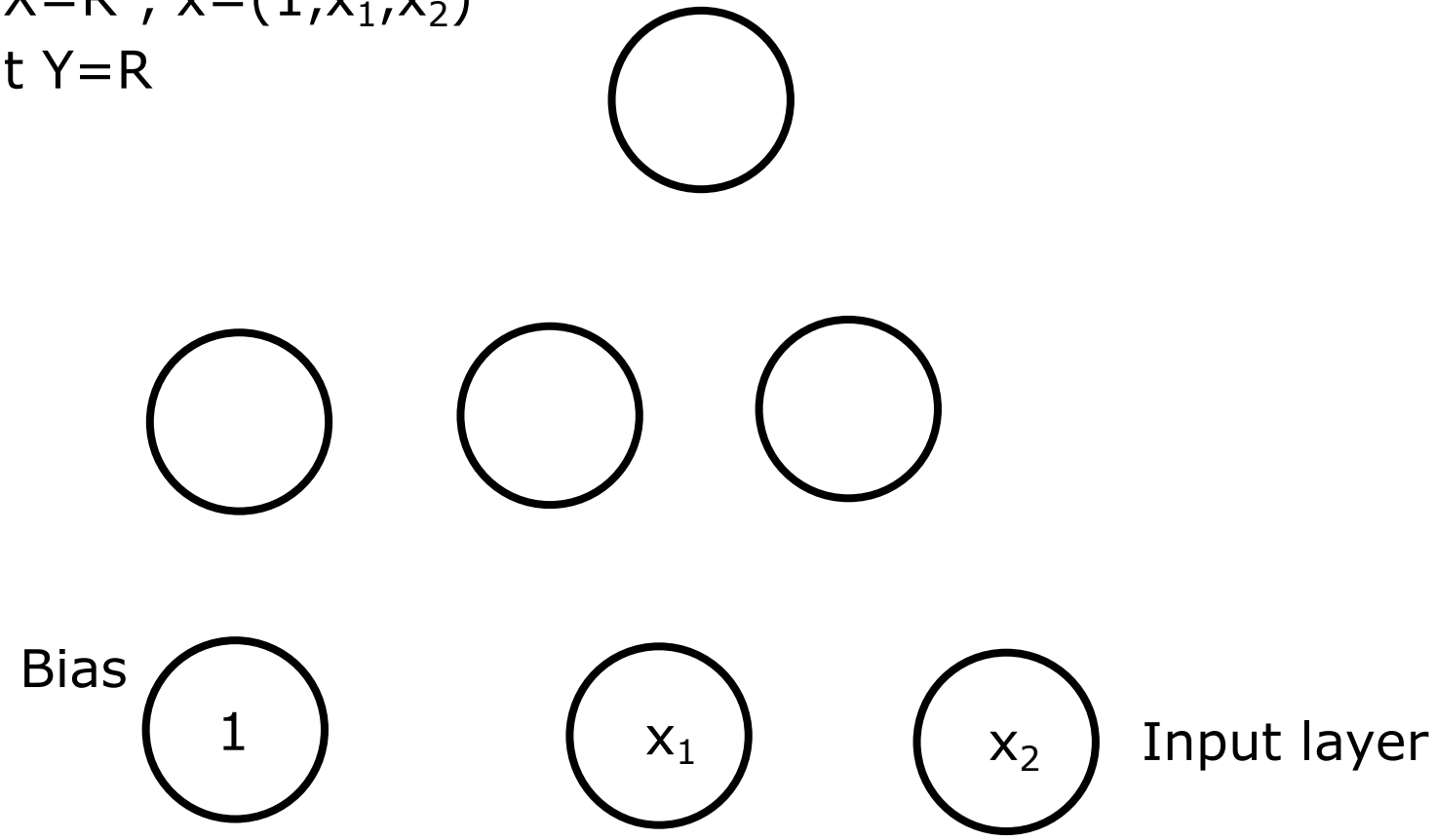
$x=(1,2,1,2)^T$

$w=(2,4,1,-3)^T$



Neural Nets

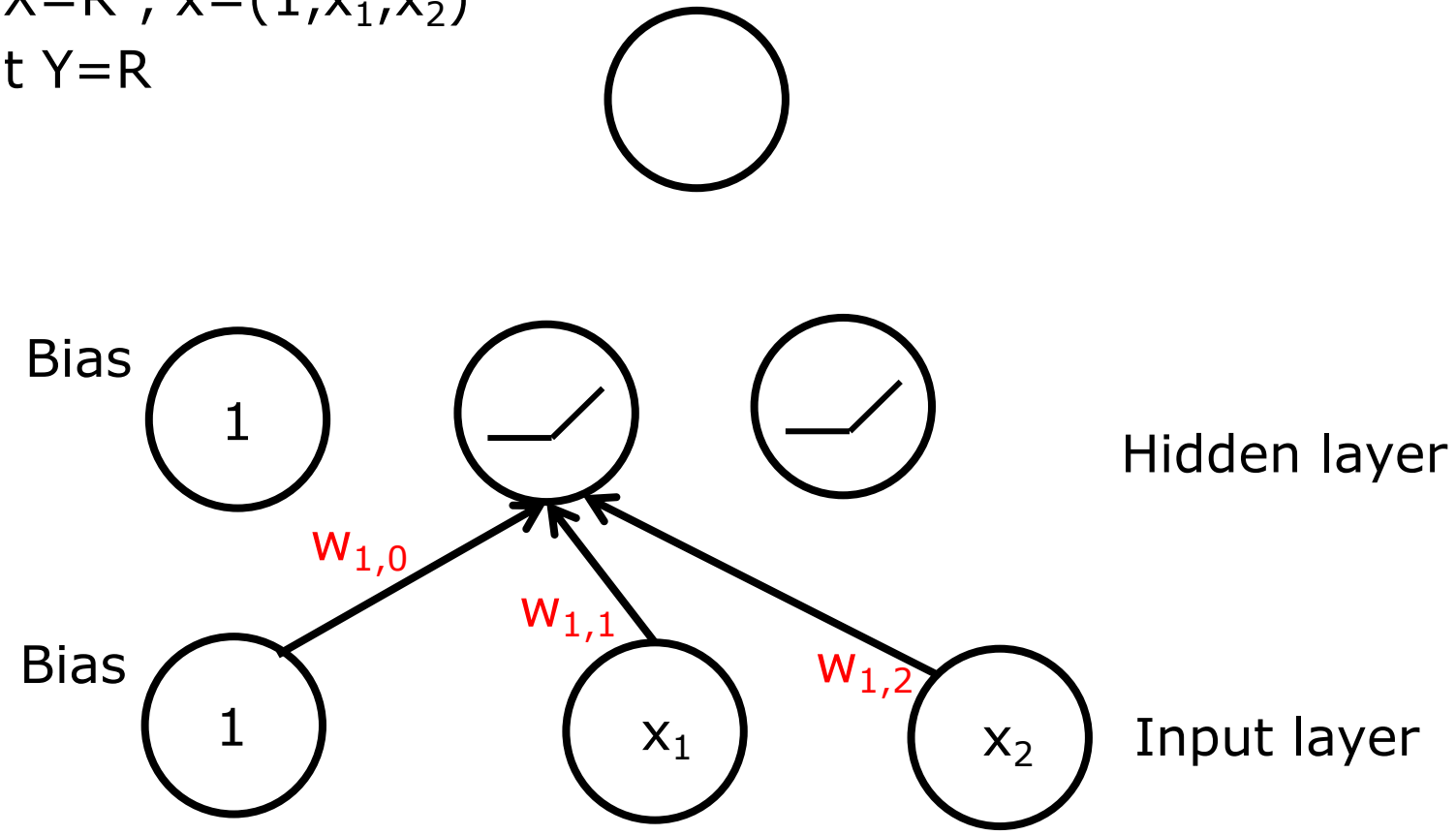
- Feed Forward Neural Net:
  - Input $X=R^2$, $x=(1,x_1,x_2)^T$
  - Output $Y=R$



Neural Nets

- Feed Forward Neural Net:
  - Input $X=R^2$, $x=(1,x_1,x_2)^T$
  - Output $Y=R$

$w_1=(w_{1,0},w_{1,1},w_{1,2})^T$

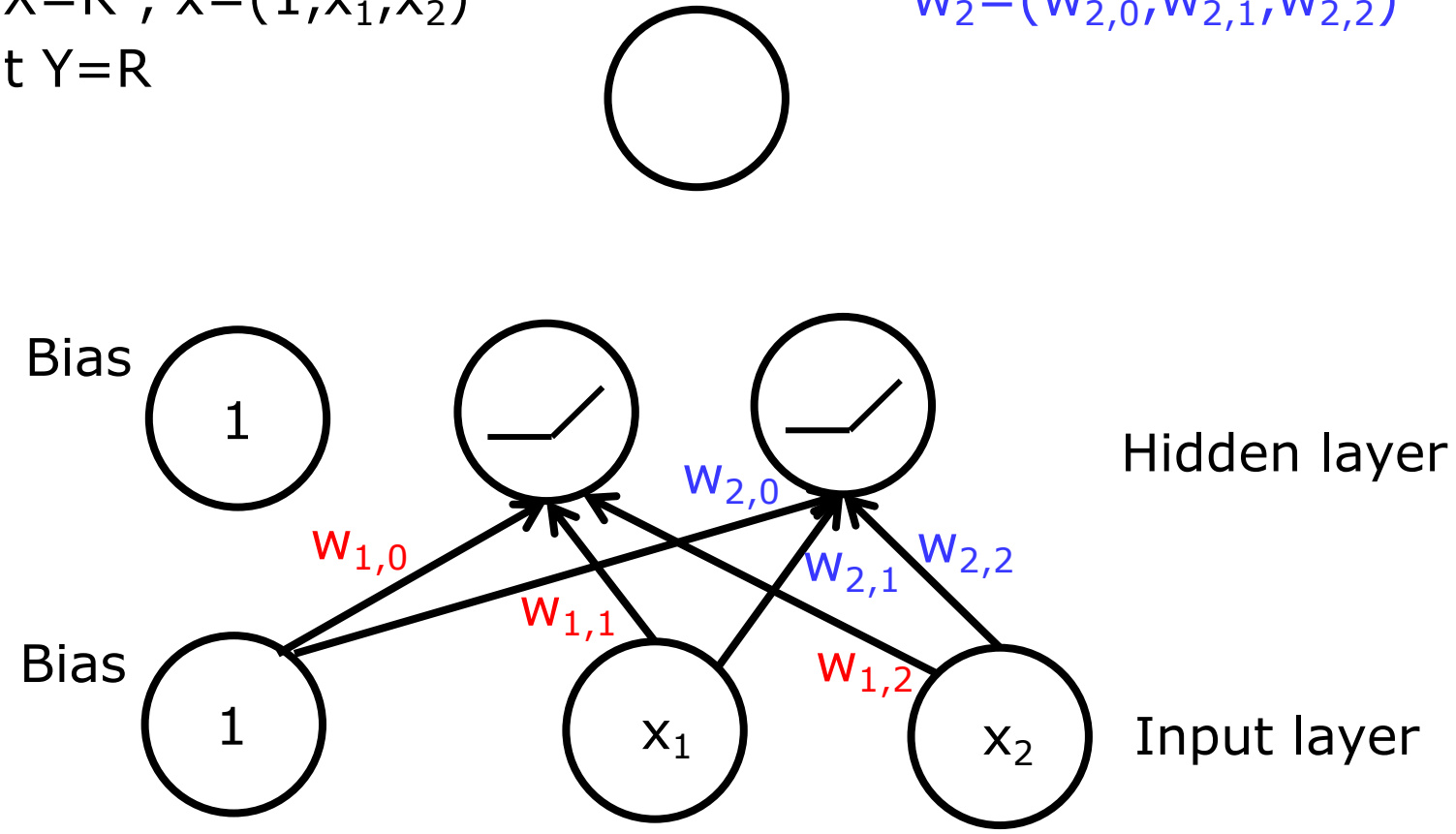


Neural Nets

- Feed Forward Neural Net:
  - Input $X=R^2$, $x=(1,x_1,x_2)^T$
  - Output $Y=R$

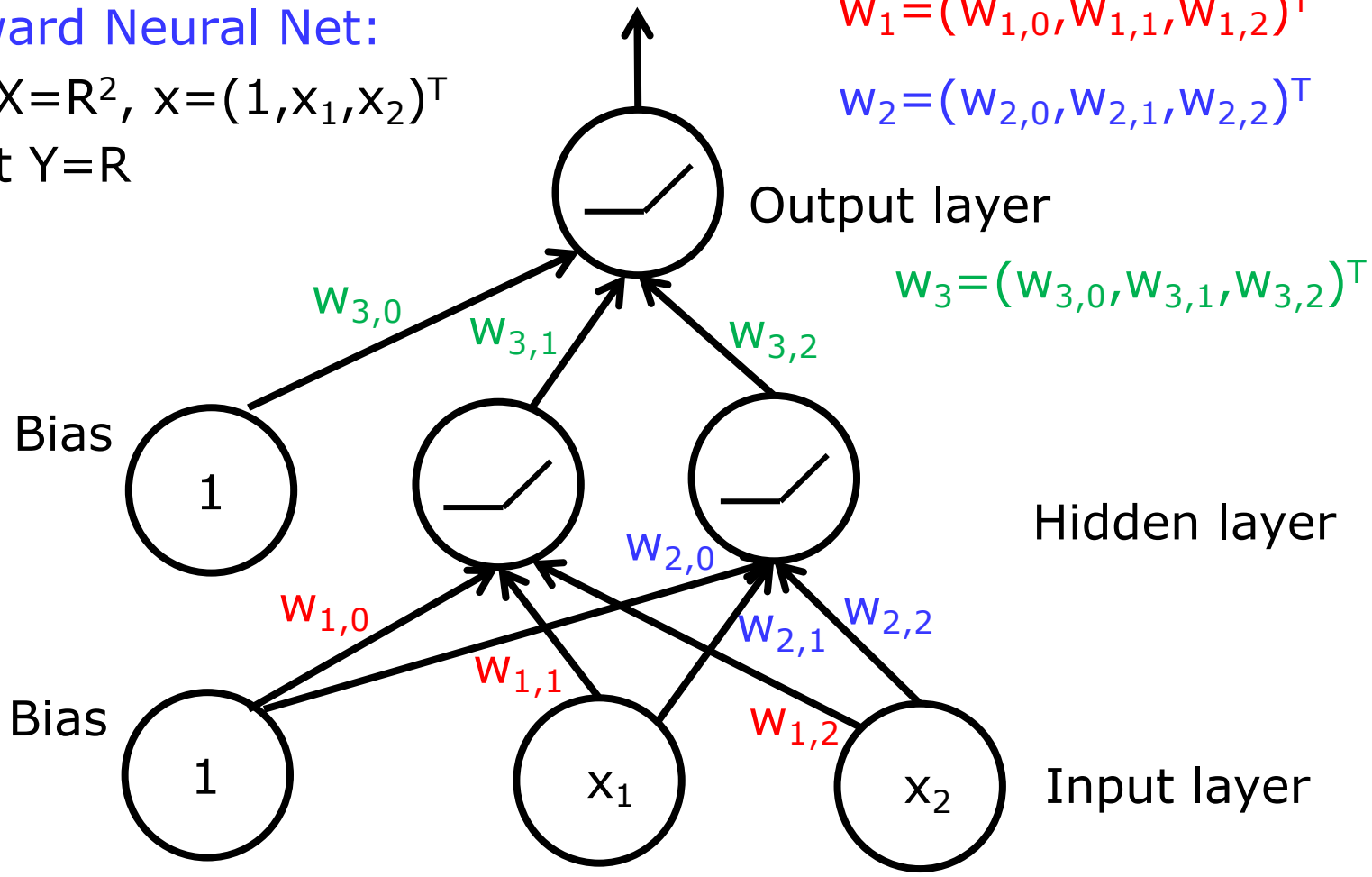
$w_1=(w_{1,0},w_{1,1},w_{1,2})^T$

$w_2=(w_{2,0},w_{2,1},w_{2,2})^T$



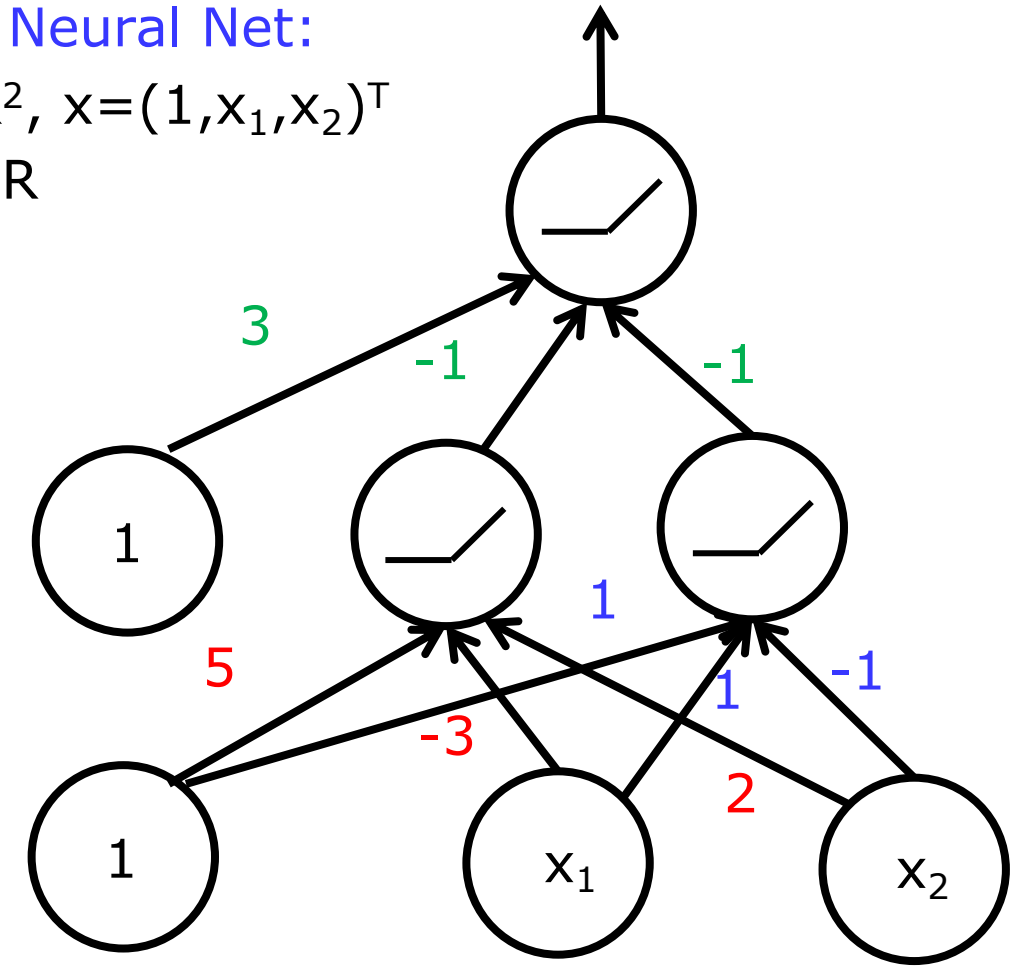
Neural Nets

- Feed Forward Neural Net:
  - Input $X=R^2$, $x=(1,x_1,x_2)^T$
  - Output $Y=R$



Neural Nets

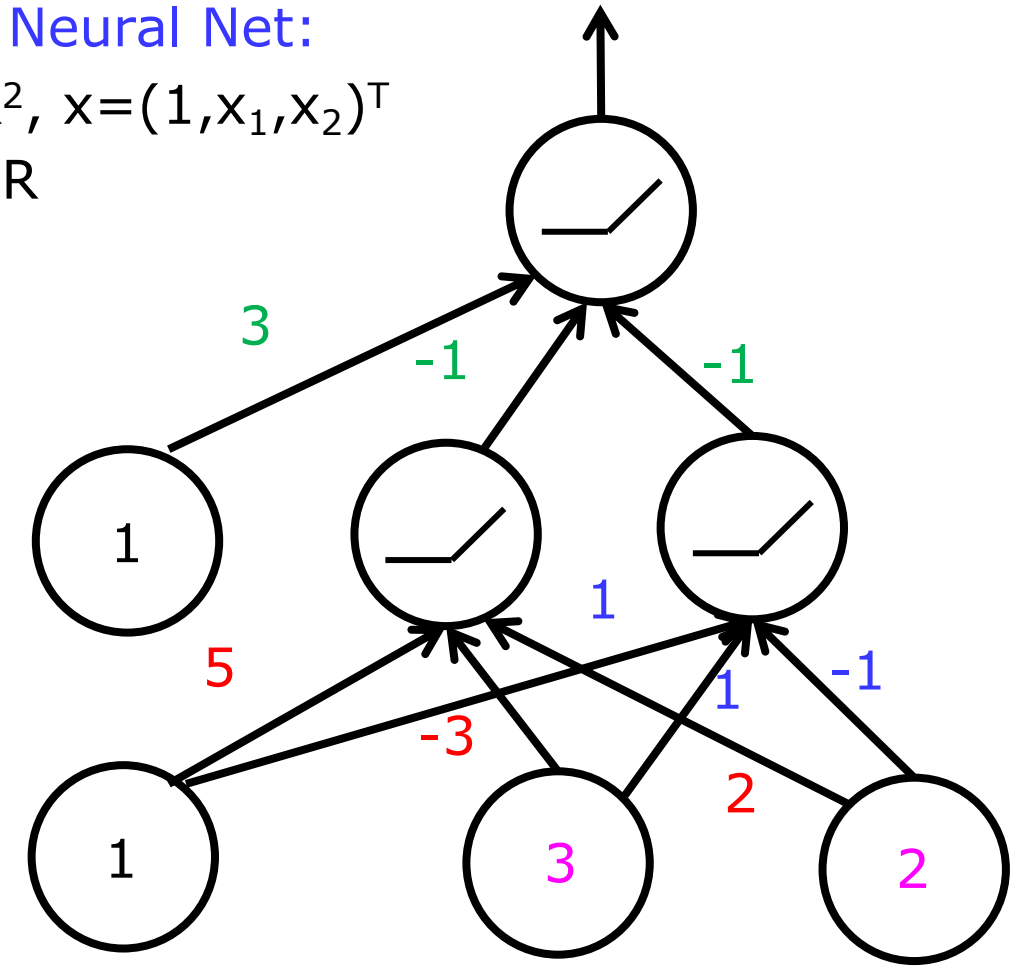
- Feed Forward Neural Net:
  - Input $X=R^2$, $x=(1,x_1,x_2)^T$
  - Output $Y=R$



Neural Nets

$x=(1,3,2)^T$

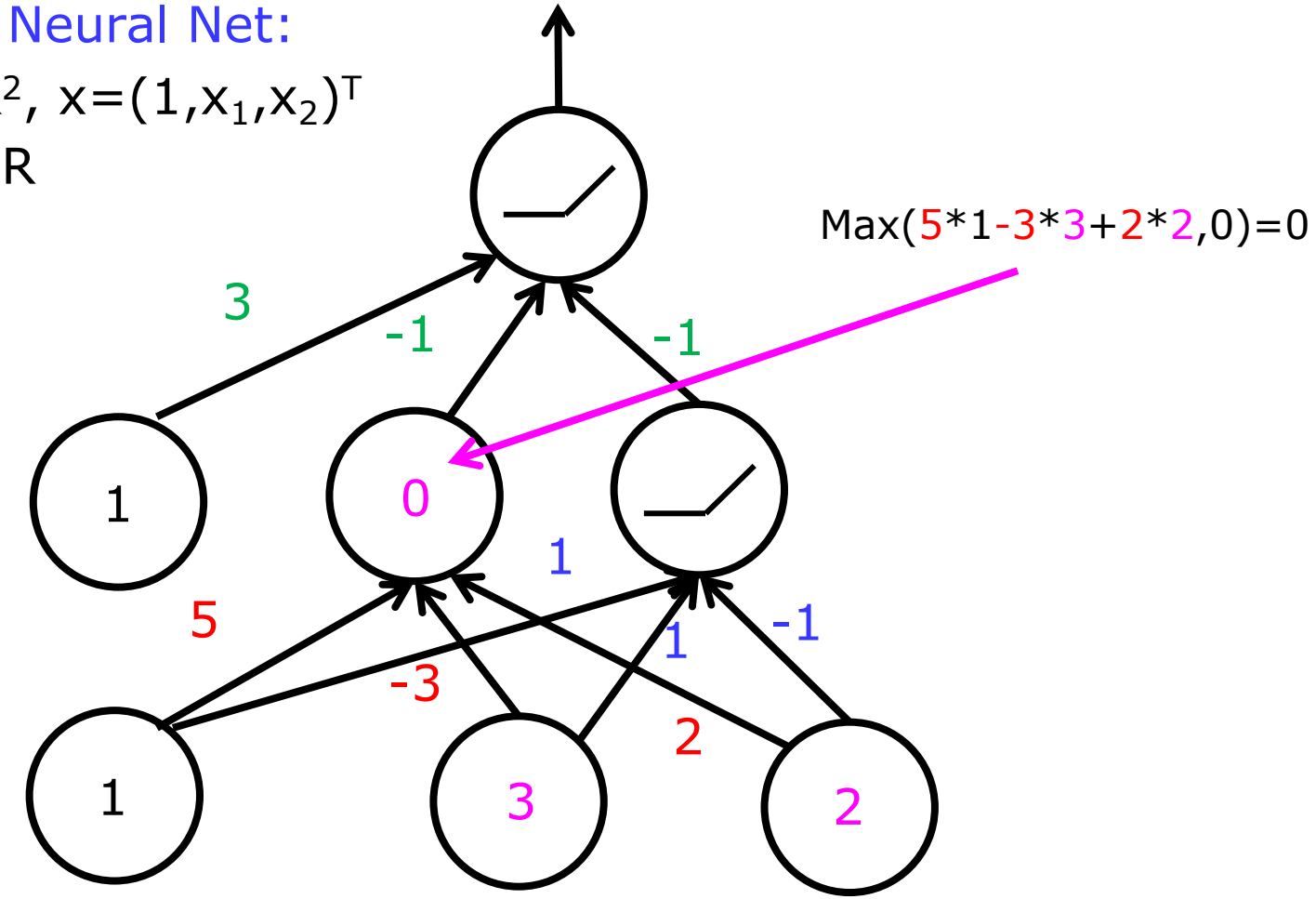
- Feed Forward Neural Net:
  - Input $X=R^2$, $x=(1,x_1,x_2)^T$
  - Output $Y=R$



Neural Nets

$x=(1,3,2)^T$

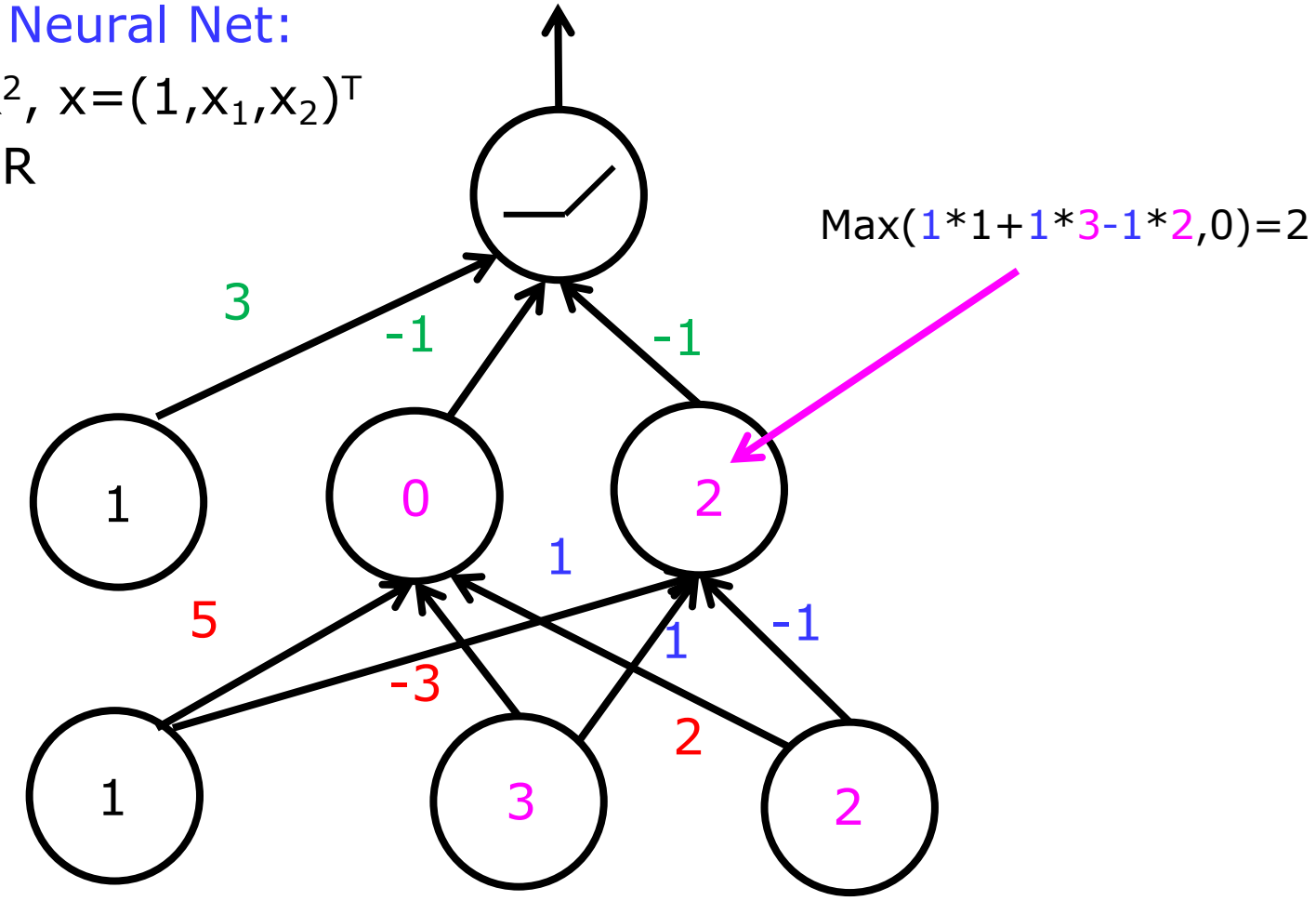
- Feed Forward Neural Net:
  - Input $X=R^2$, $x=(1,x_1,x_2)^T$
  - Output $Y=R$



Neural Nets

$x=(1,3,2)^T$

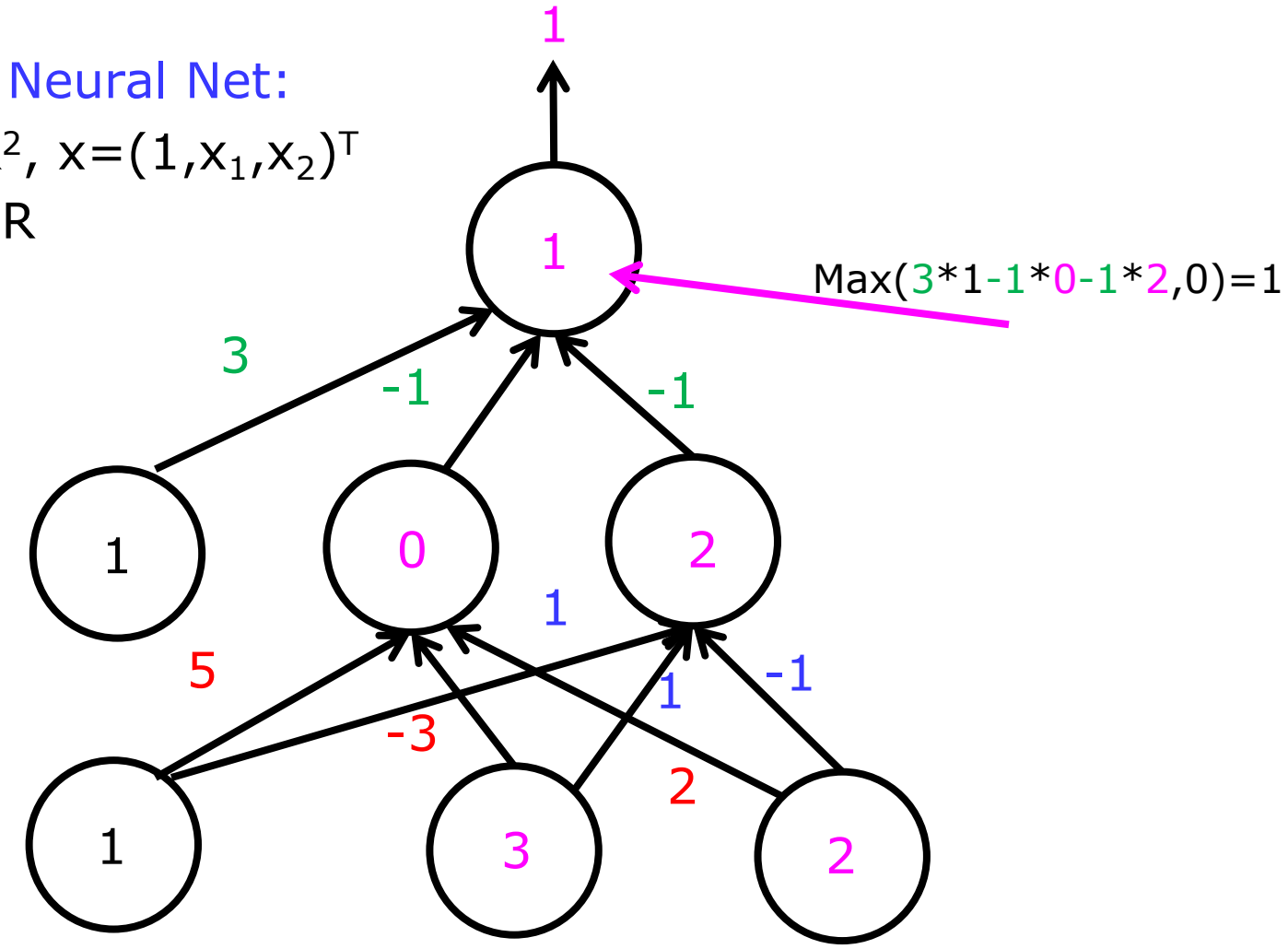
- Feed Forward Neural Net:
  - Input $X=R^2$, $x=(1,x_1,x_2)^T$
  - Output $Y=R$



Neural Nets

$x=(1,3,2)^T$

- Feed Forward Neural Net:
  - Input $X=R^2$, $x=(1,x_1,x_2)^T$
  - Output $Y=R$



Neural Nets

- Feed Forward Neural Net:
  - Feed Forward: All edges point ”forward”.
  - Formally: Acyclic directed graph.

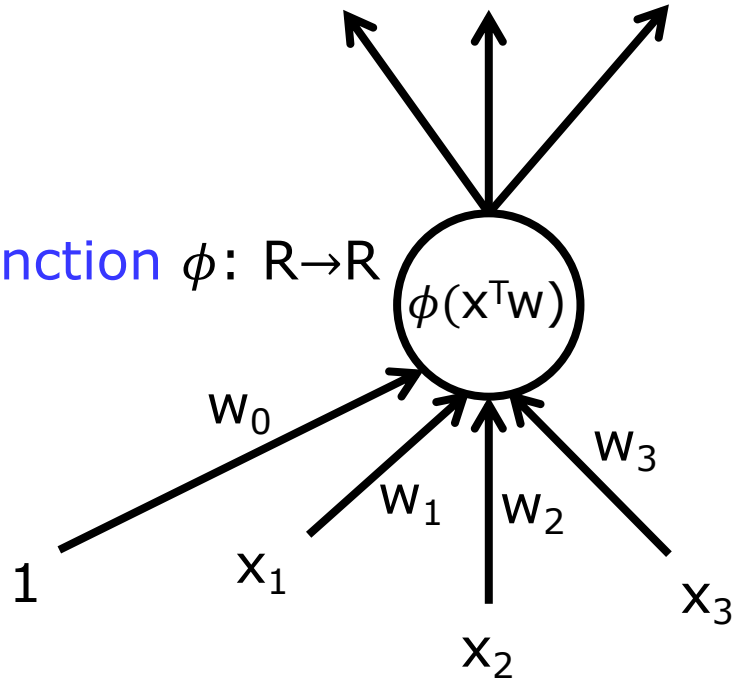
  - There are other networks with back-edges (recurrent neural nets).
  - More complicated, not in this course.

Neural Networks

Activation Functions

- Activations:
  - Exist many choices.
  - Many work roughly equally well.

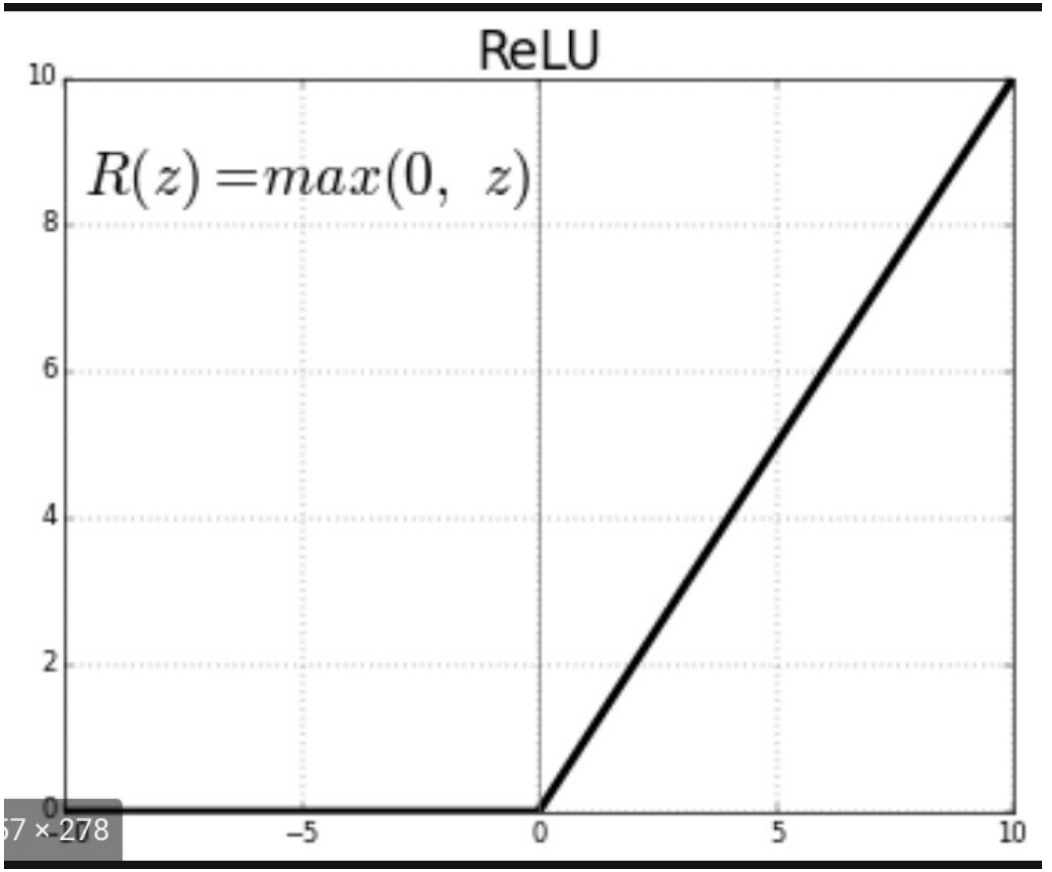
Activation function ϕ: R→R



ReLU

- Rectified Linear Unit

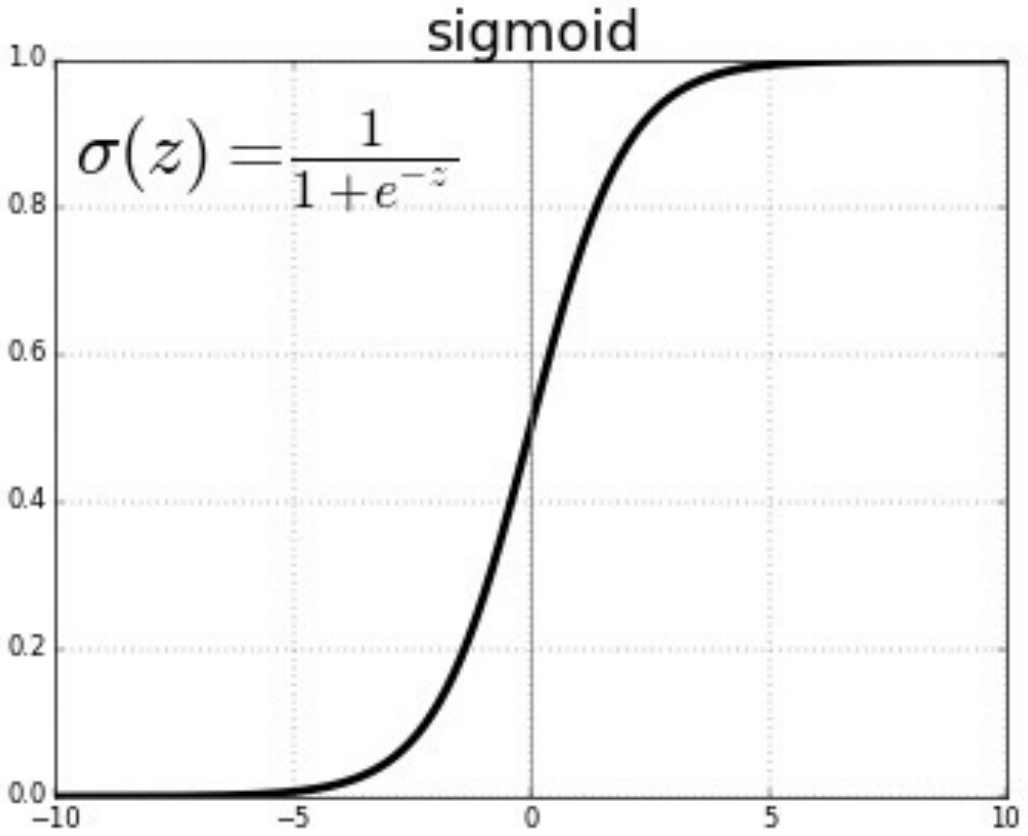
- ReLU $\phi(z)$
  - Linear for z>0.
  - Constant for z≤0.
  - Non-linear function.
- ”Fires” for z>0.



Sigmoid

- Logistic Sigmoid

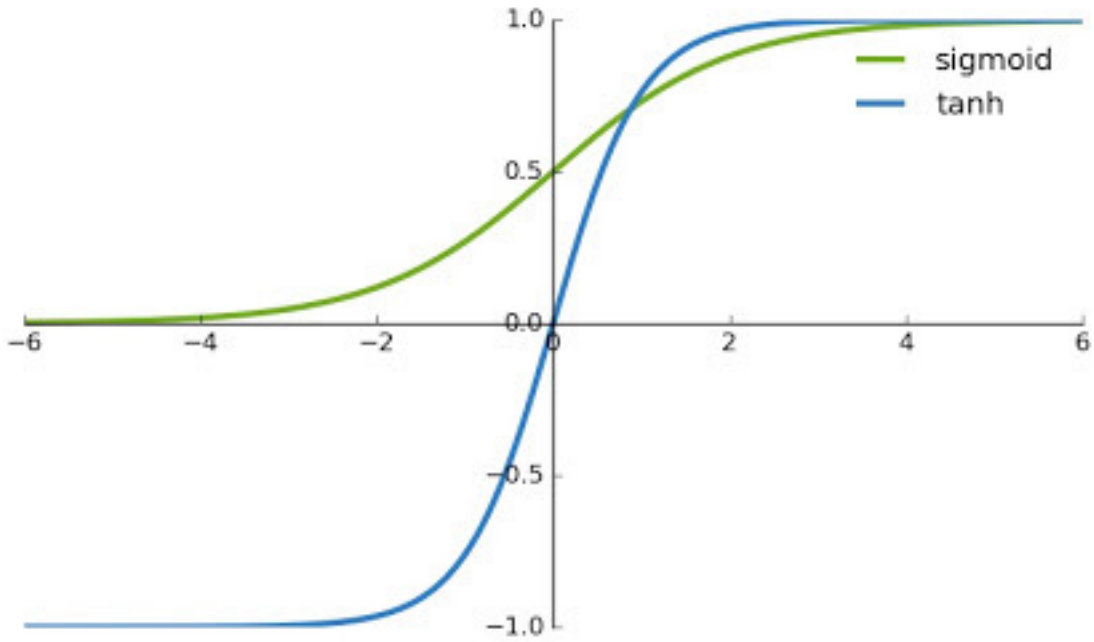
- Sigmoid $\sigma(z)$
  - ”Fires” as z becomes big enough.



Hyperbolic Tangent

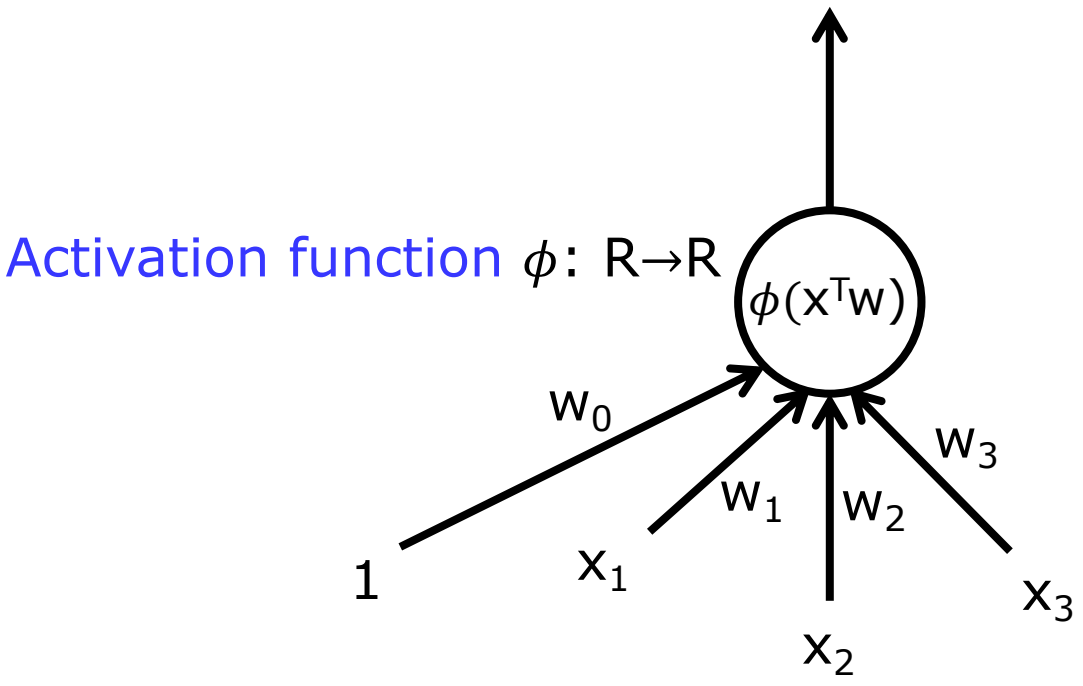
- Hyperbolic Tangent

- $\tanh(z)$
  - $\tanh(z) = 2\sigma(2z) - 1$



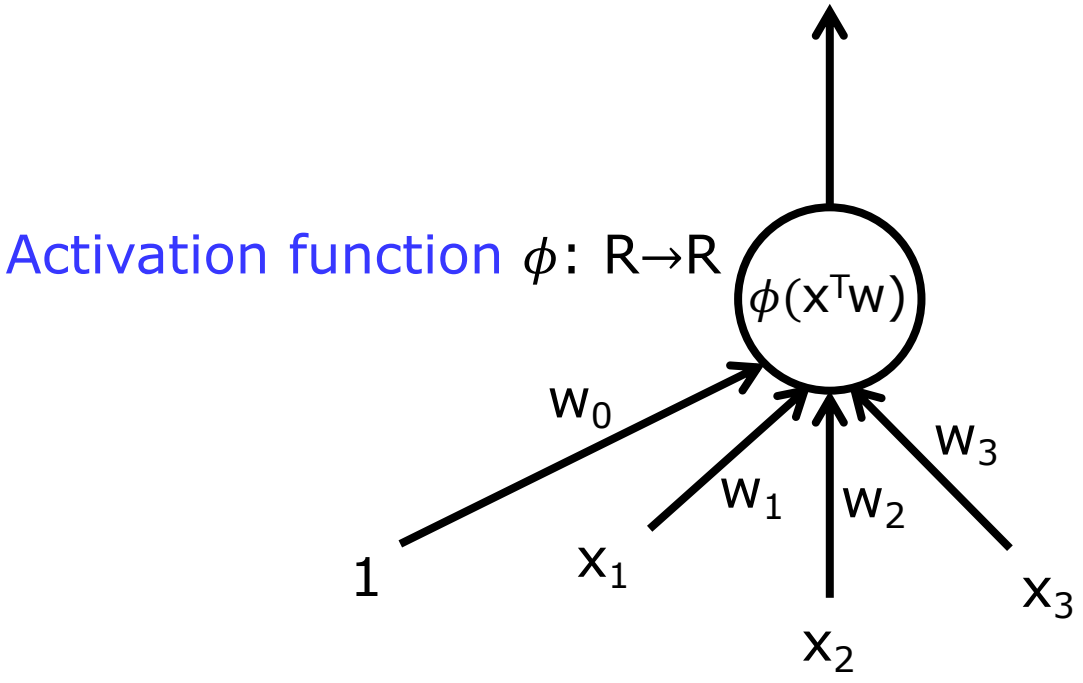
Activation Functions

- Output Neurons
  - Often have their own special activation functions.



Activation Functions

- Regression Output Neurons
  - For regression problems with least squares loss $L(y',y)=(y-y')^2$, often use Identity $I(z)=z$.



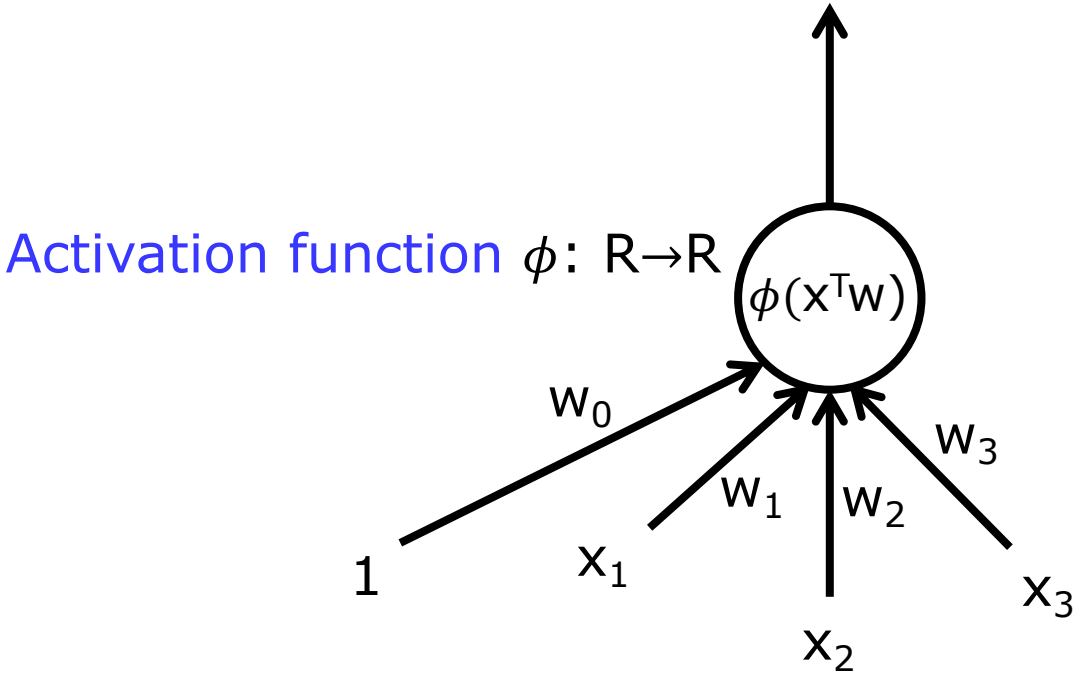
No hidden layers:

Exactly linear regression as compute $w^T x$.

Linear regression special case of neural nets.

Activation Functions

- Binary Classication Output Neurons
  - For binary classification problems with negative log-likelihood (like logistic regression), often use Sigmoid $\sigma(z)$.



No hidden layers:

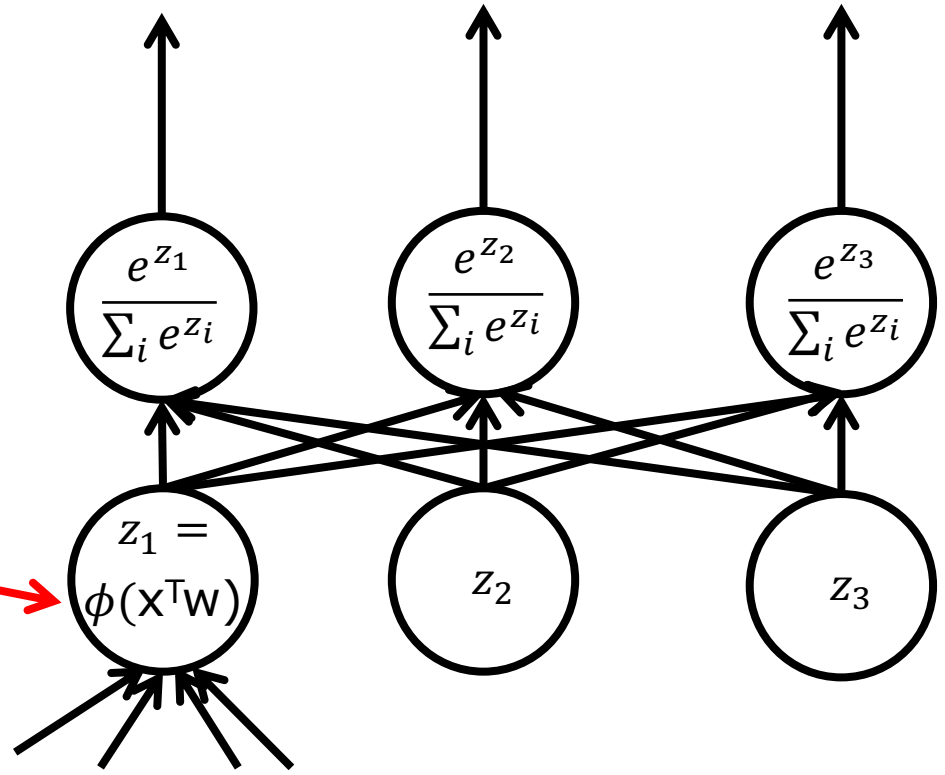
Exactly logistic regression as compute $\sigma(w^T x)$.

Logistic regression special case of neural nets.

Activation Functions

- Multiclass Classication Output Neurons
  - For classification problems with negative log-likelihood (like multinomial/softmax regression), often use Softmax neurons $\text{softmax}(z)_i$.

Activation function ϕ: $R^k \to R$



Often ϕ =Identity

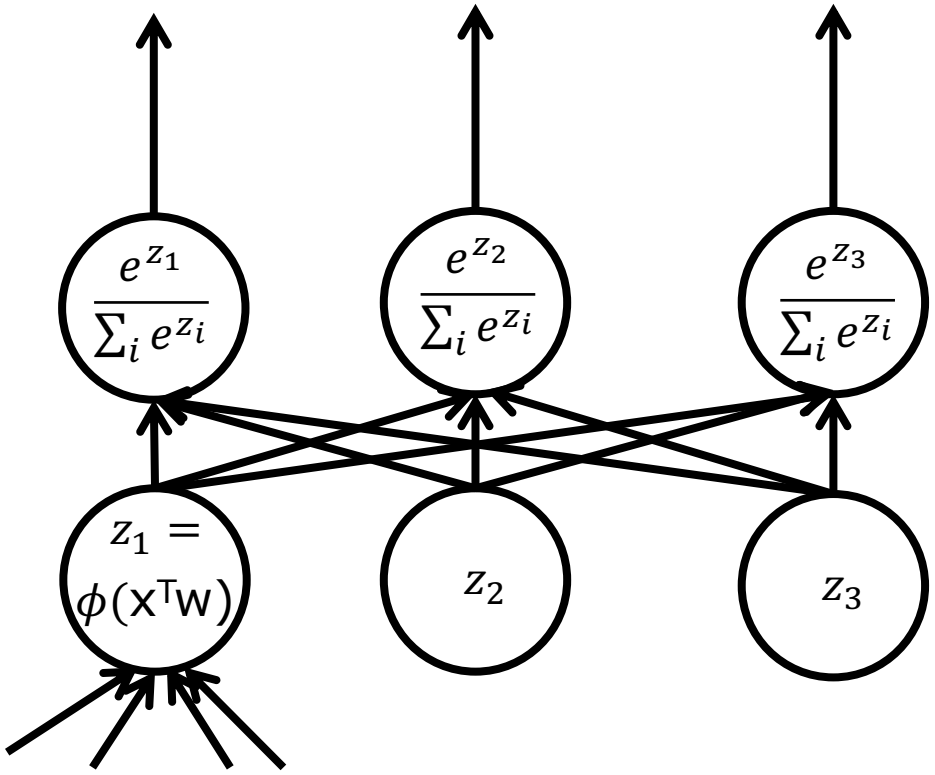
Activation Functions

- Multiclass Classication Output Neurons
  - For classification problems with negative log-likelihood (like multinomial/softmax regression), often use Softmax neurons $\text{softmax}(z)_i$.

One hidden layer:

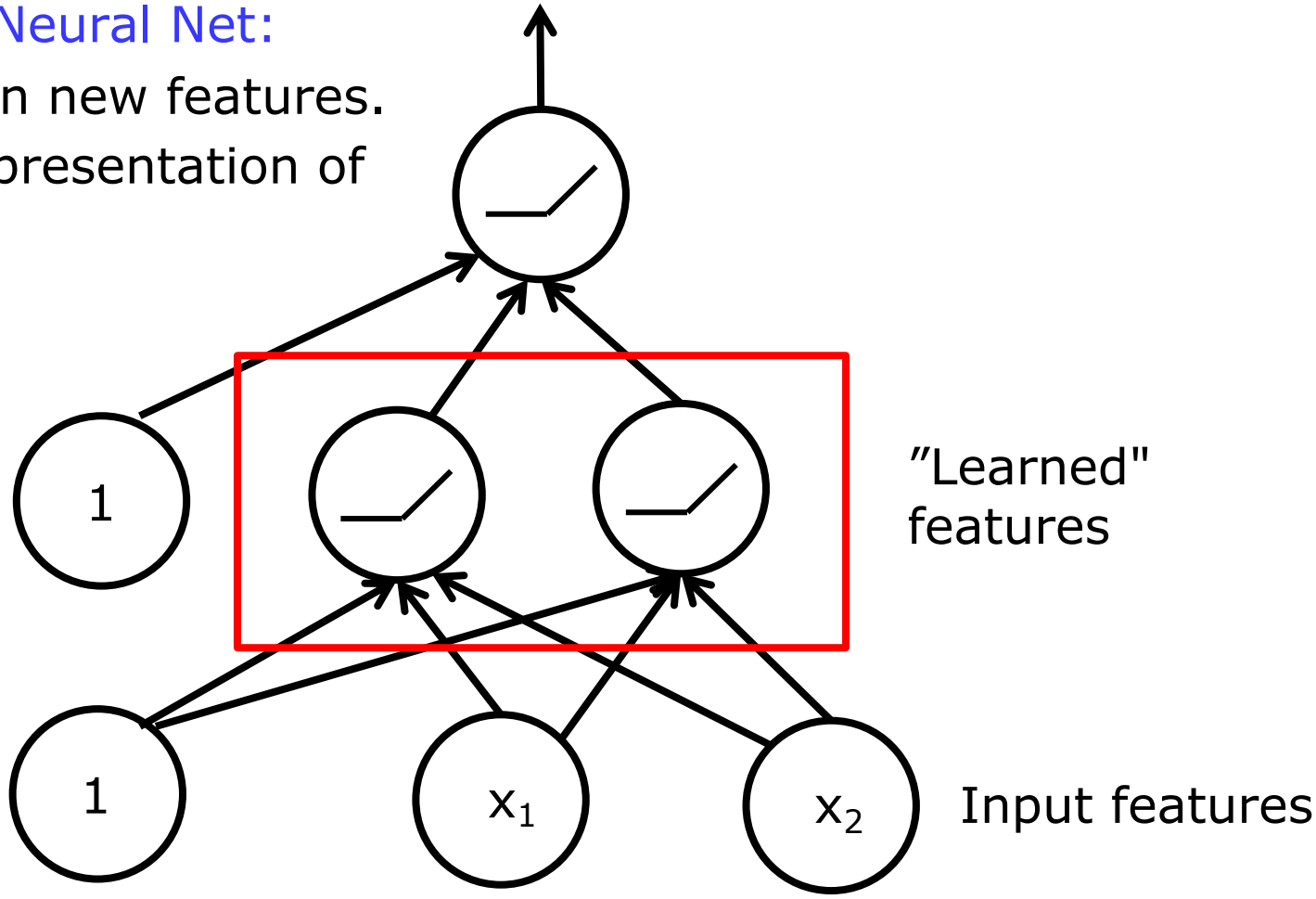
Exactly multinomial/soft max regression.

Multinomial regression special case of neural nets.



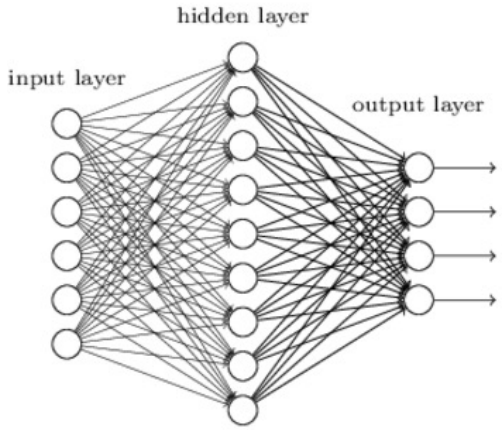
Feature Transform

- Feed Forward Neural Net:
  - Layers learn new features.
  - Another representation of input.



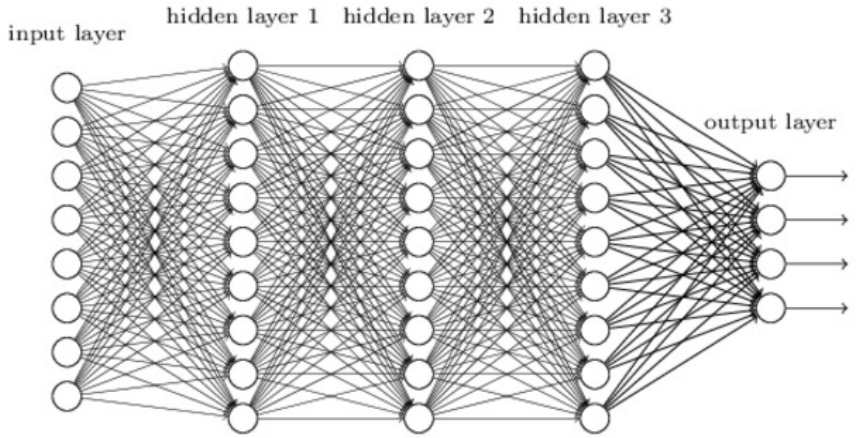
Feature Transform

"Non-deep" feedforward neural network



Learned feaures

Deep neural network



"Low level" features

"High level" features

Deep Learning

- Machine learning models based on learning multiple levels of abstraction.

Deep Learning: Automating Feature Discovery

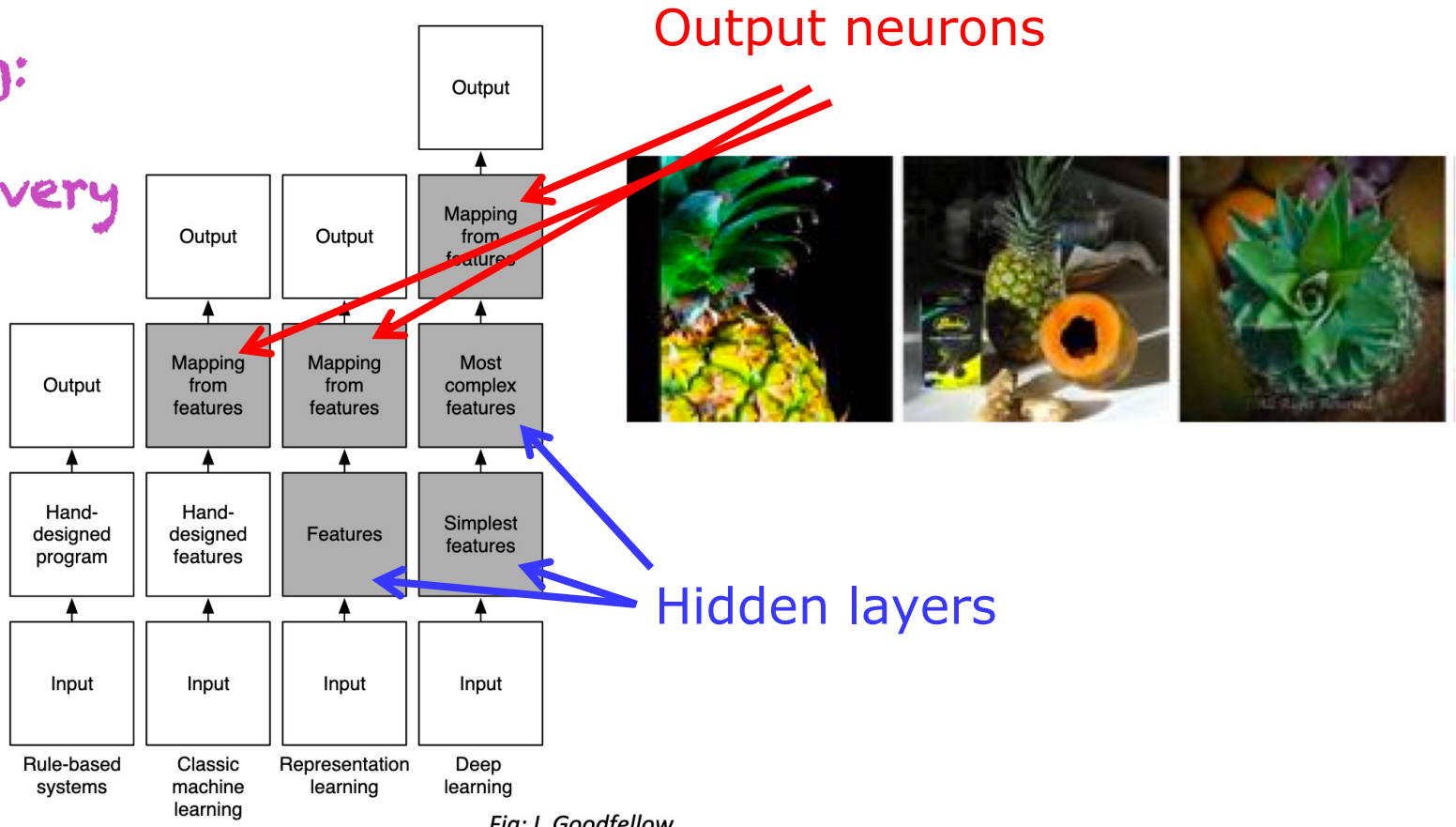


Fig: I. Goodfellow

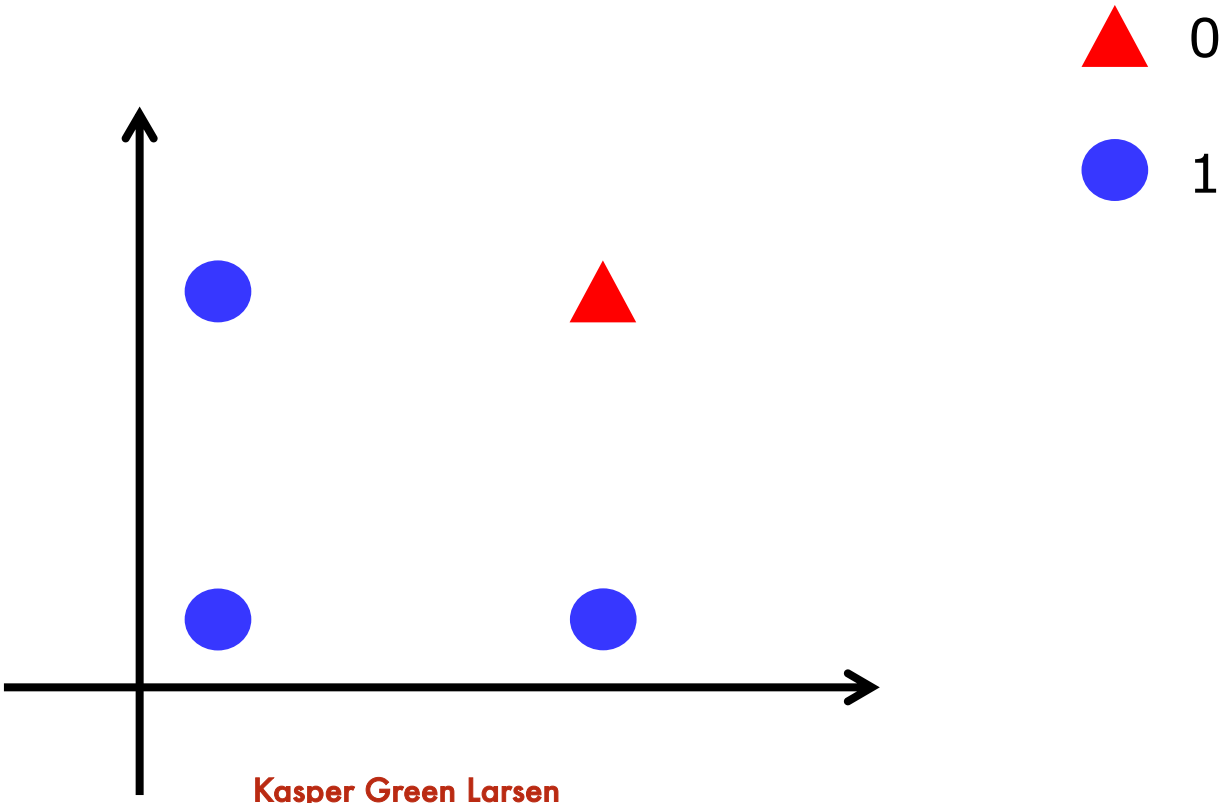
Powerful

- Can compute ”anything”.
- NAND gate, for x_1,x_2 in {0,1}:
  - NAND(x_1,x_2)=0 if $x_1=x_2=1$, otherwise NAND(x_1,x_2)=1.

- From CS courses:
  - Any logical gate can be constructed from NAND gates.
  - Universal.

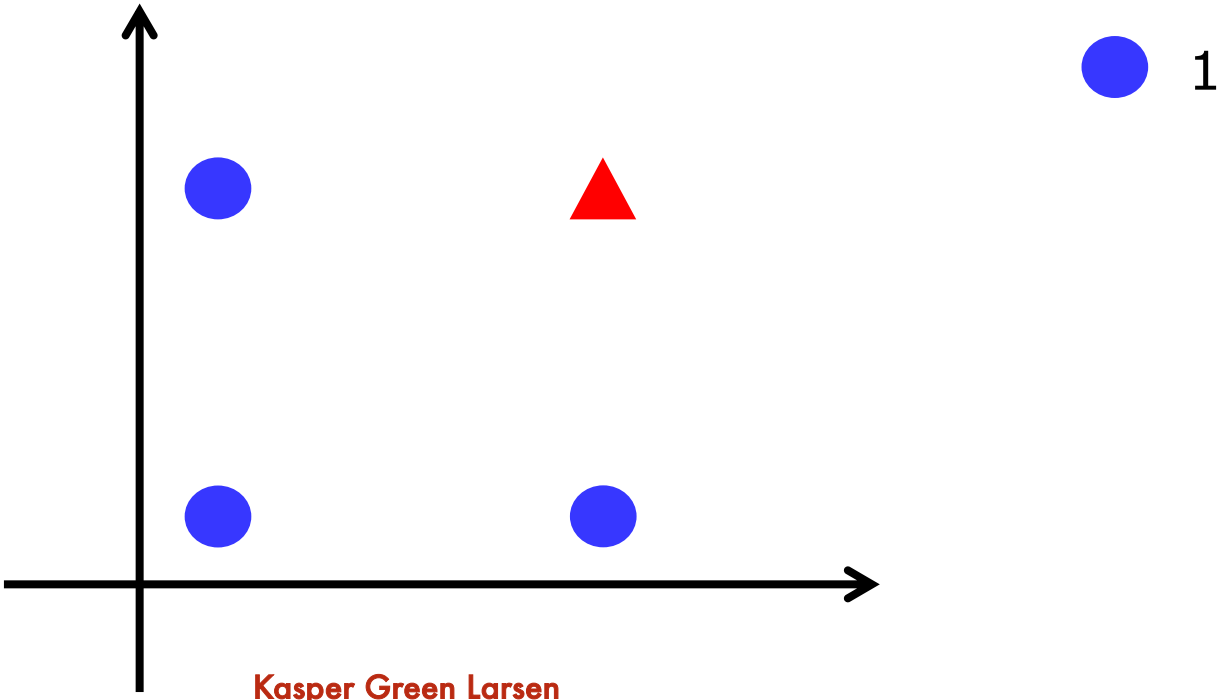
NAND

- Can compute ”anything”.
- NAND gate, for x_1,x_2 in {0,1}:
  - NAND(x_1,x_2)=0 if $x_1=x_2=1$, otherwise NAND(x_1,x_2)=1.



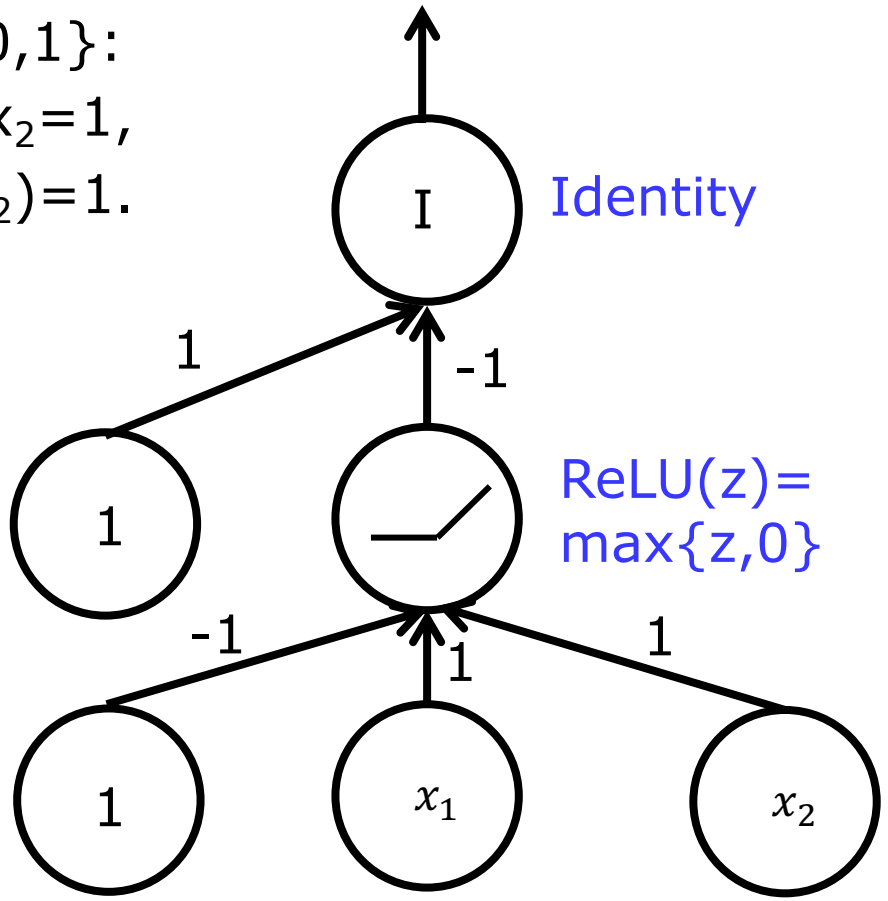
NAND

- Not Linear:
  - If $(w_0,w_1,w_2)^T(1,0,0)=1$ then bias $w_0=1$.
  - If $(1,w_1,w_2)^T(1,1,0)=1$ then $w_1=0$.
  - If $(1,0,w_2)^T(1,0,1)=1$ then $w_2=0$.
  - But $(1,0,0)^T(1,1,1)=1$



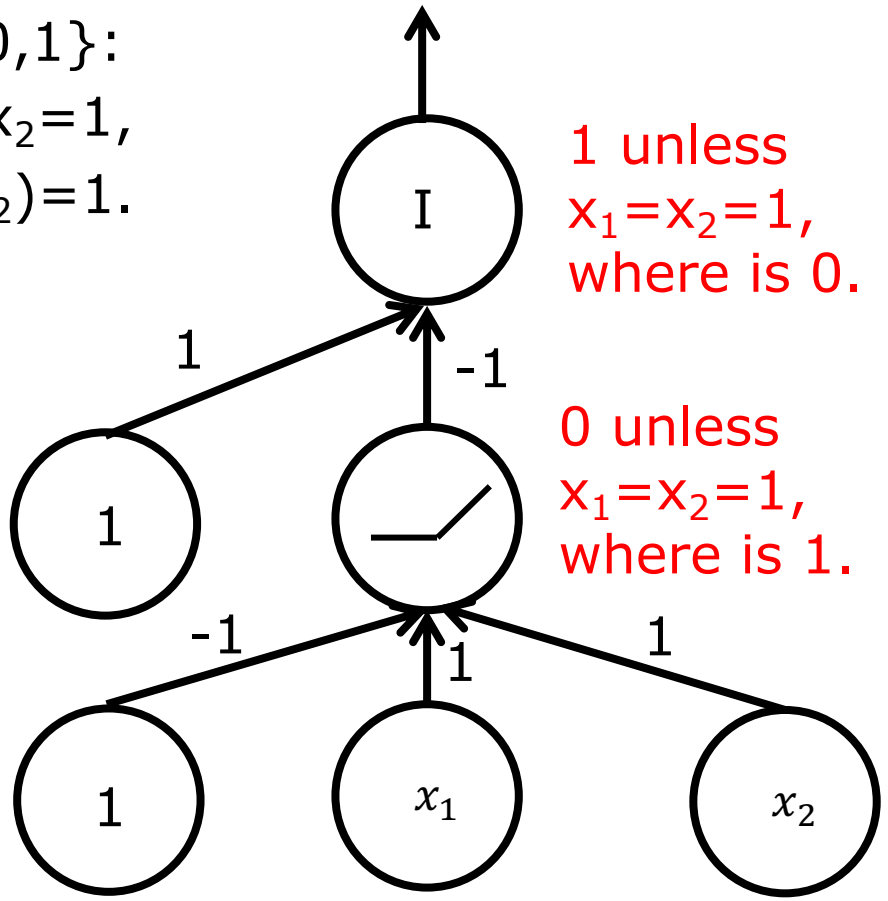
NAND

- Can compute ”anything”.
- NAND gate, for x_1,x_2 in {0,1}:
  - NAND(x_1,x_2)=0 if $x_1=x_2=1$,
    otherwise NAND(x_1,x_2)=1.



NAND

- Can compute ”anything”.
- NAND gate, for x_1,x_2 in {0,1}:
  - NAND(x_1,x_2)=0 if $x_1=x_2=1$, otherwise NAND(x_1,x_2)=1.



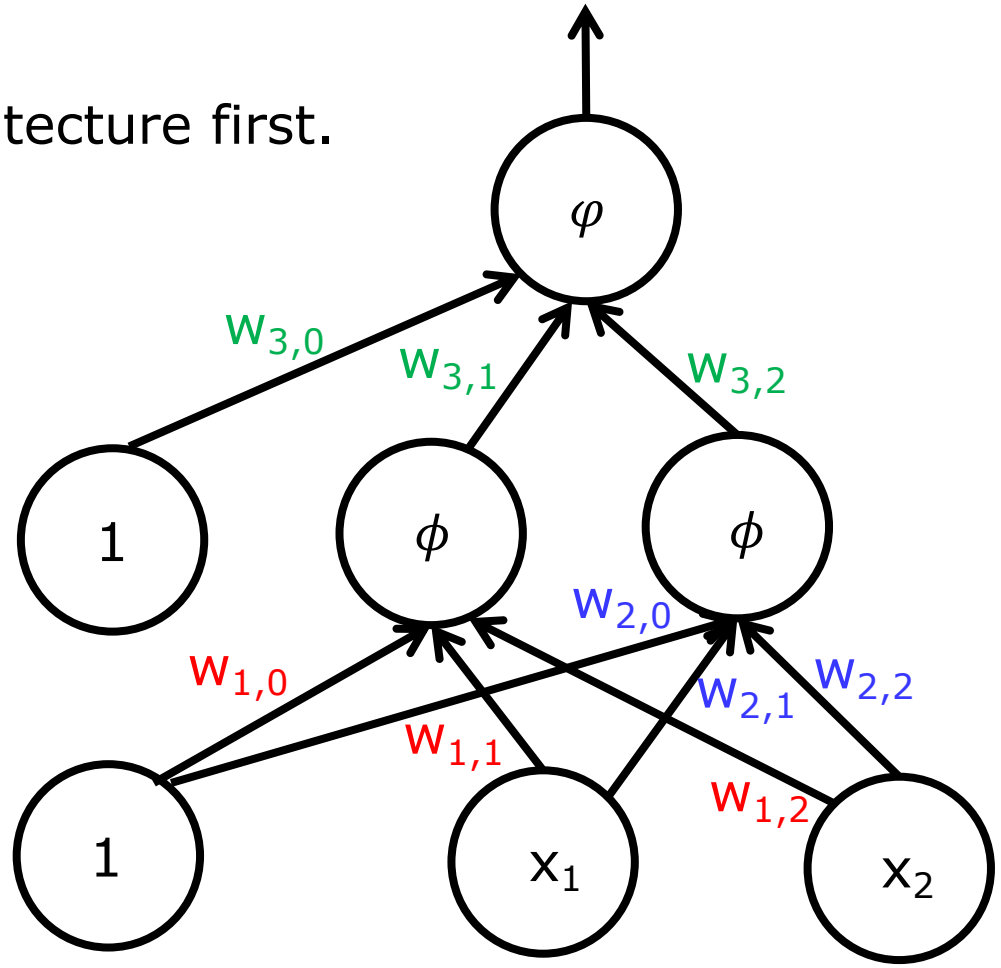
Powerful

- Universal Approximation Theorem
  - Neural Net with 1 hidden layer and ReLU can approximate ”any” function arbitrarily well given enough hidden units.
    - Not only ReLU but most non-linear activations.
  - May require exponentially many hidden units.
  - Not really important for us, but interesting statement.

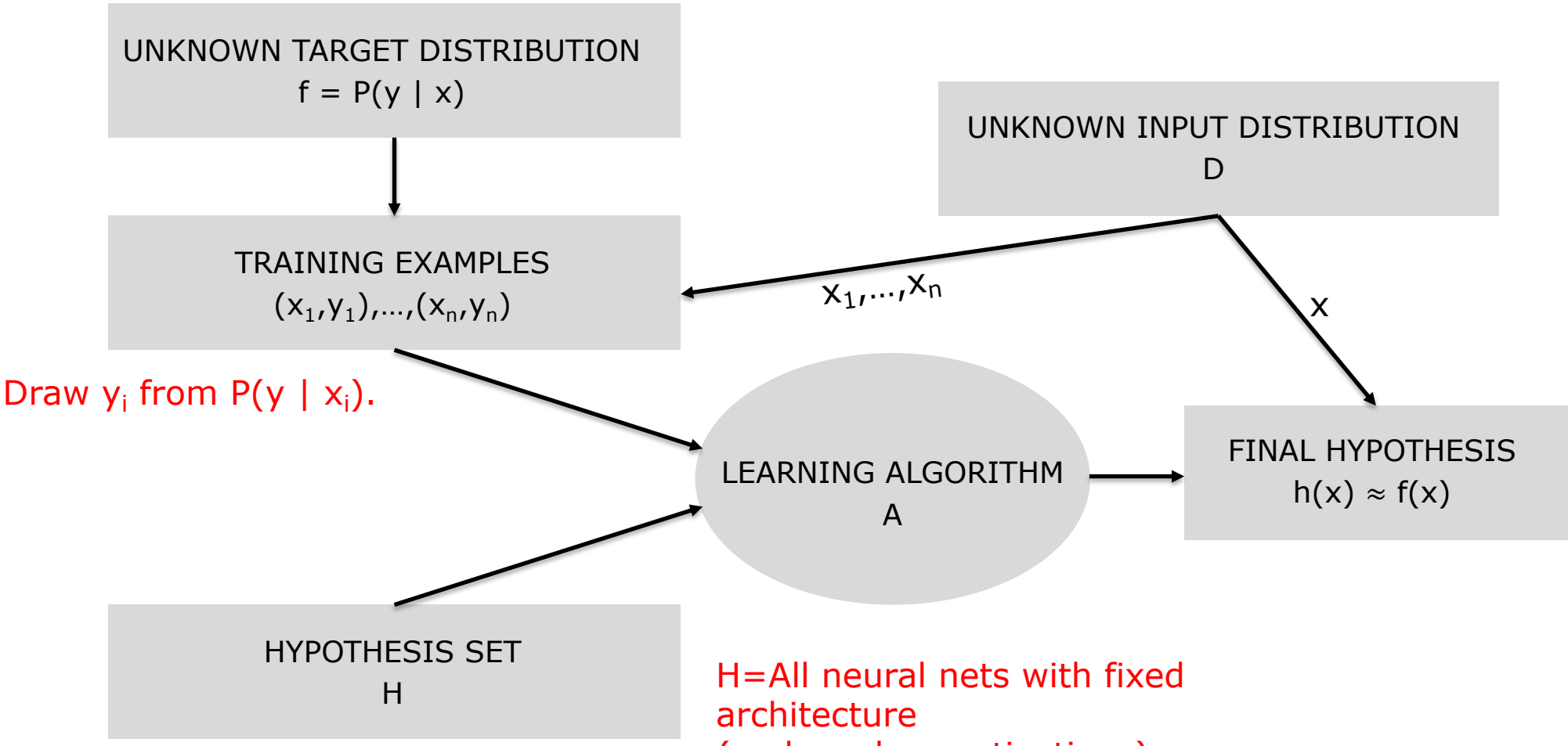
Neural Networks

Neural Net Architectures

- Feed Forward Neural Net.
  - When train, choose architecture first.
    - Nodes
    - Edges
    - Activation Functions
  - Then train weights.

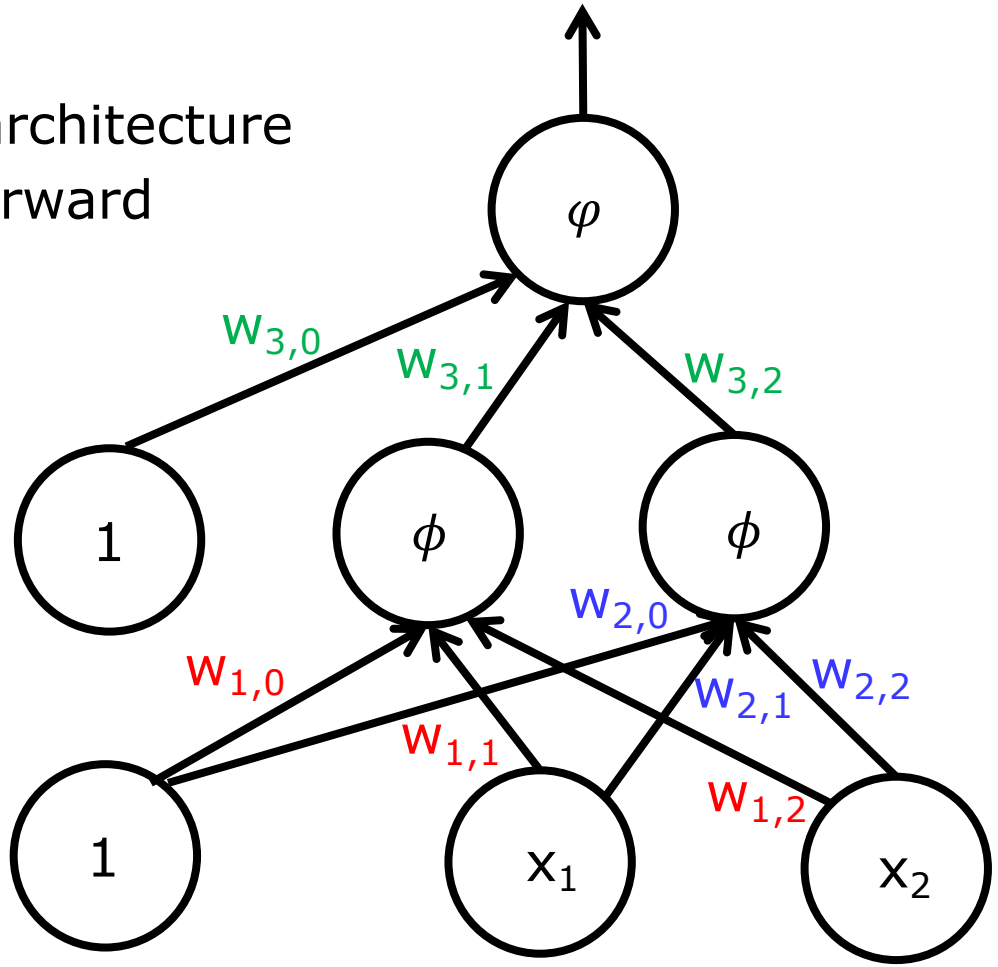


Supervised Learning



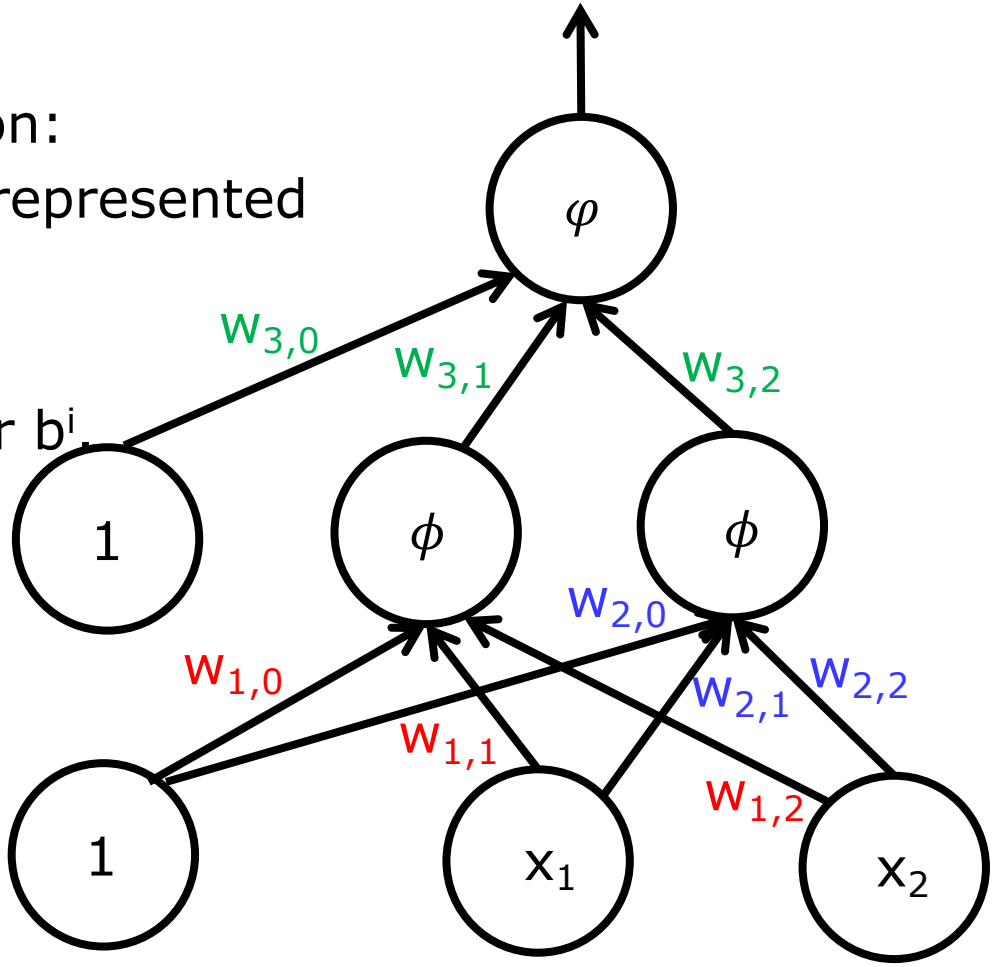
Neural Net Architectures

- Architectures
  - Most common/simplest architecture is a fully connected feed forward neural net.
  - Each hidden layer has all neurons connected to all neurons in next and previous layer.
  - See other architectures later.



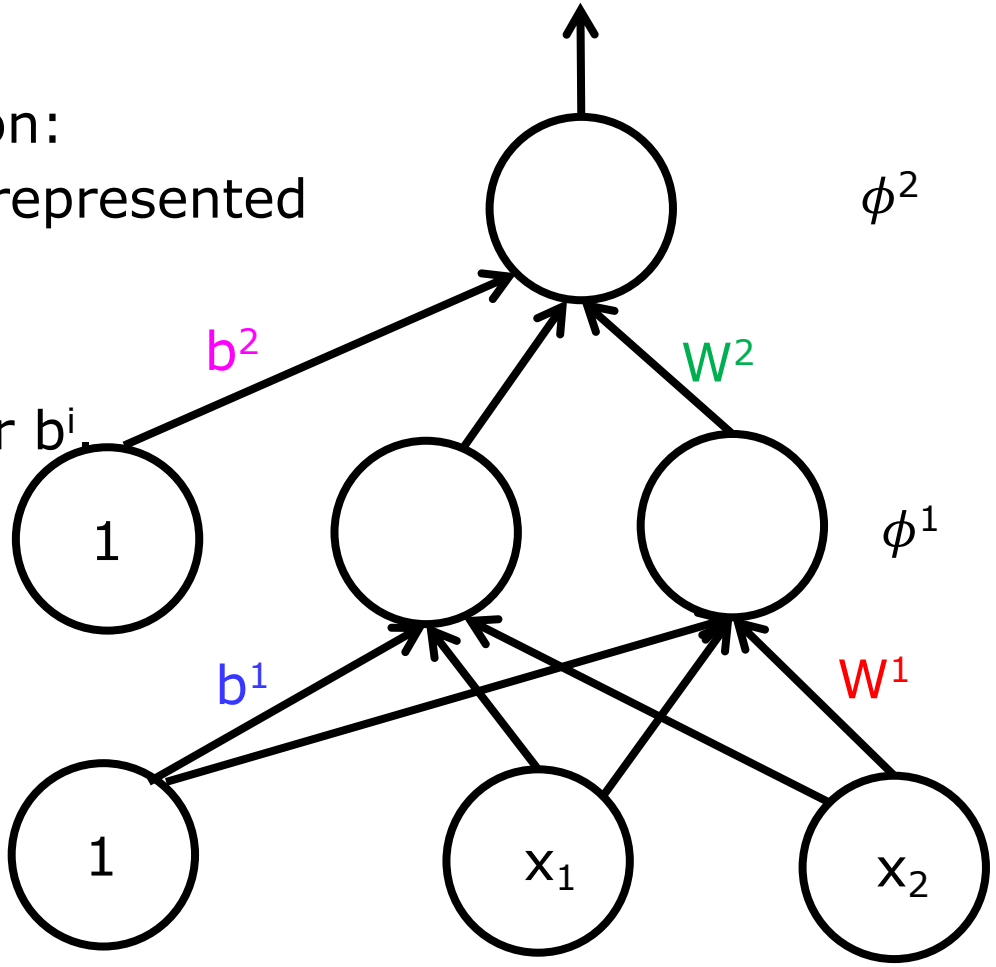
Vectorized Notation

- Fully Connected
  - Convenient representation:
    - All weights in layer i represented by a matrix W^i.
    - All biases in layer i represented by a vector b^i.
    - All neurons in layer i same activation function ϕ^i.



Vectorized Notation

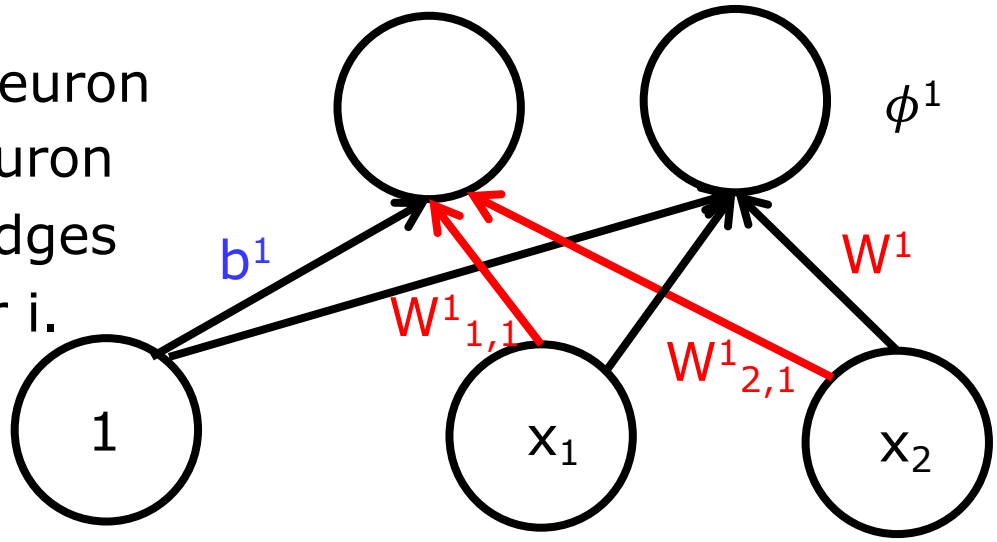
- Fully Connected
  - Convenient representation:
    - All weights in layer i represented by a matrix W^i.
    - All biases in layer i represented by a vector b^i.
    - All neurons in layer i same activation function ϕ^i.



Vectorized Notation

- Fully Connected
  - If layer i-1 has m non-bias neurons
  - Layer i has k non-bias neurons
  - W^i an mxk matrix.

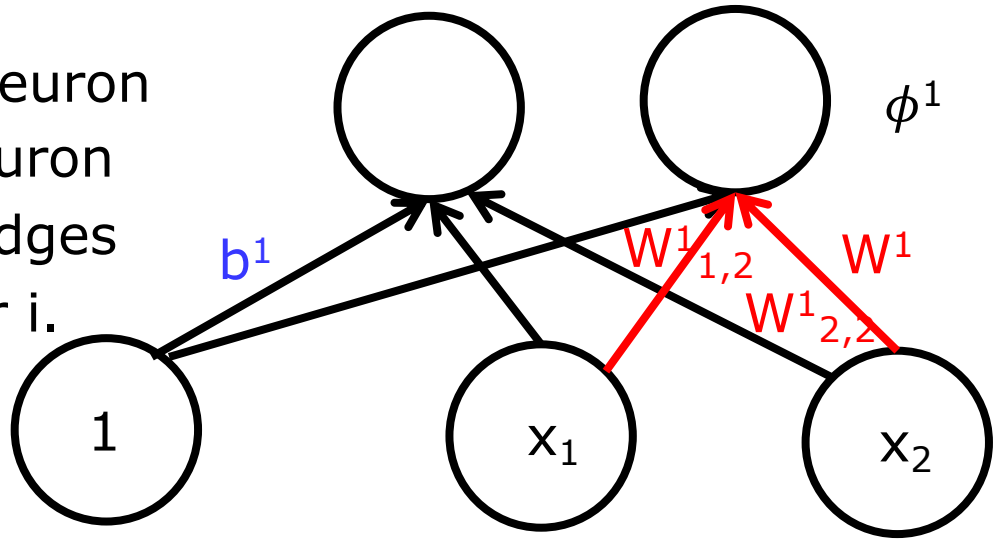
- W^i:
  - One column per layer i neuron
  - One row per layer i-1 neuron
  - Col. j gives weights on edges into j'th neuron on layer i.



Vectorized Notation

- Fully Connected
  - If layer i-1 has m non-bias neurons
  - Layer i has k non-bias neurons
  - W^i an mxk matrix.

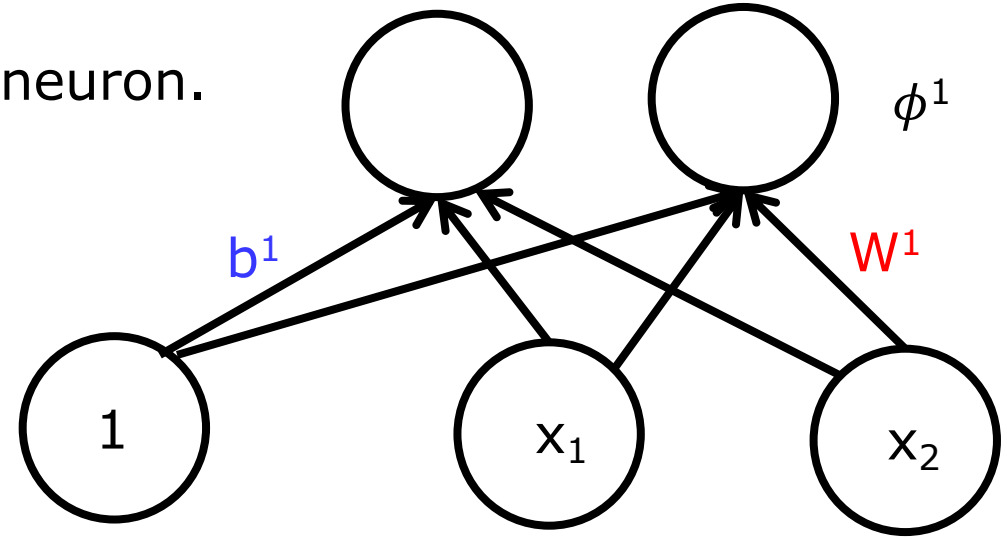
- W^i:
  - One column per layer i neuron
  - One row per layer i-1 neuron
  - Col. j gives weights on edges into j'th neuron on layer i.



Vectorized Notation

- Fully Connected
  - Layer i has k non-bias neurons
  - b^i is a row-vector with k entries.

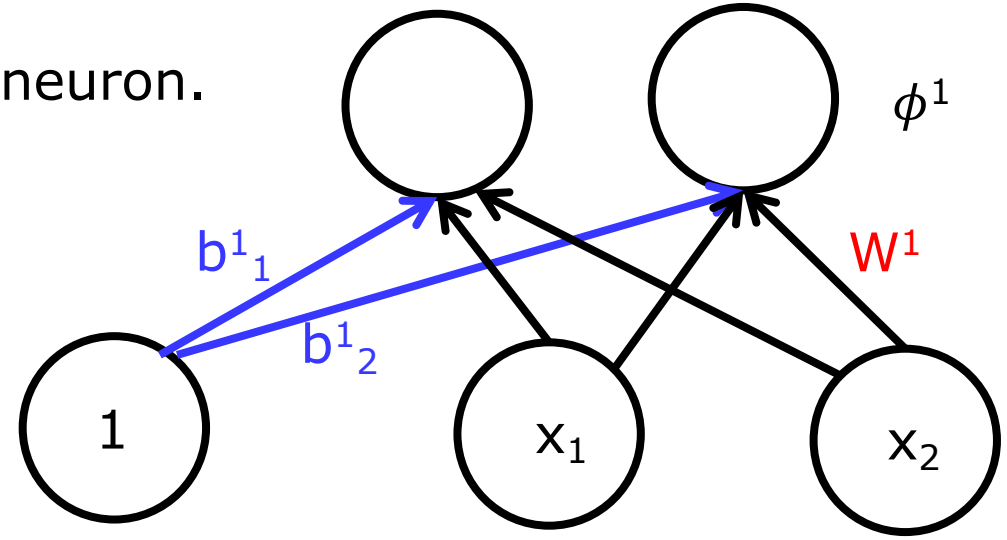
- b^i:
  - One entry per neuron.
  - Entry j gives bias of j'th neuron.



Vectorized Notation

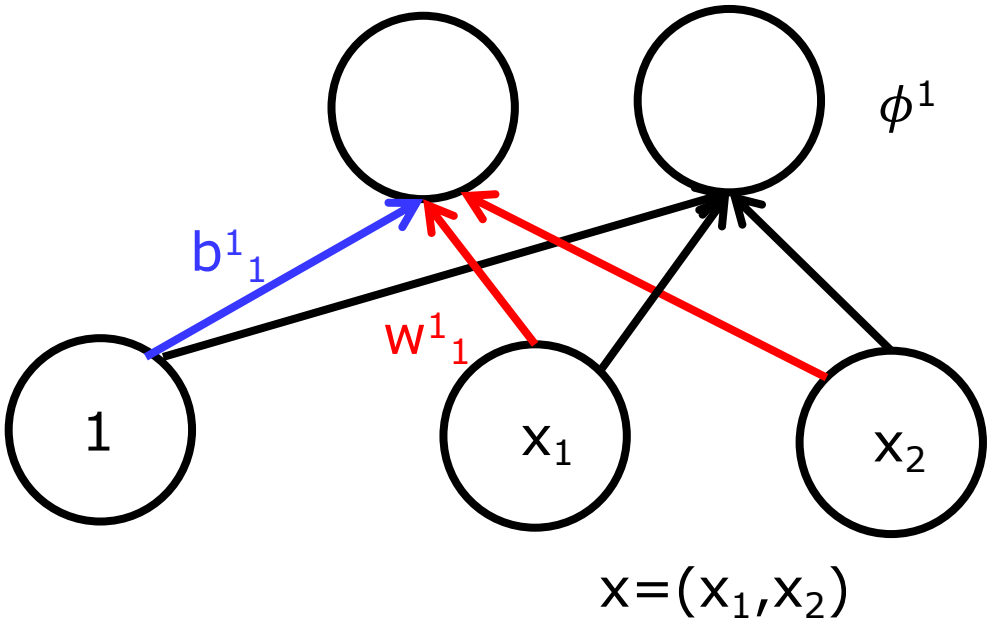
- Fully Connected
  - Layer i has k non-bias neurons
  - b^i is a row-vector with k entries.

- b^i:
  - One entry per neuron.
  - Entry j gives bias of j'th neuron.



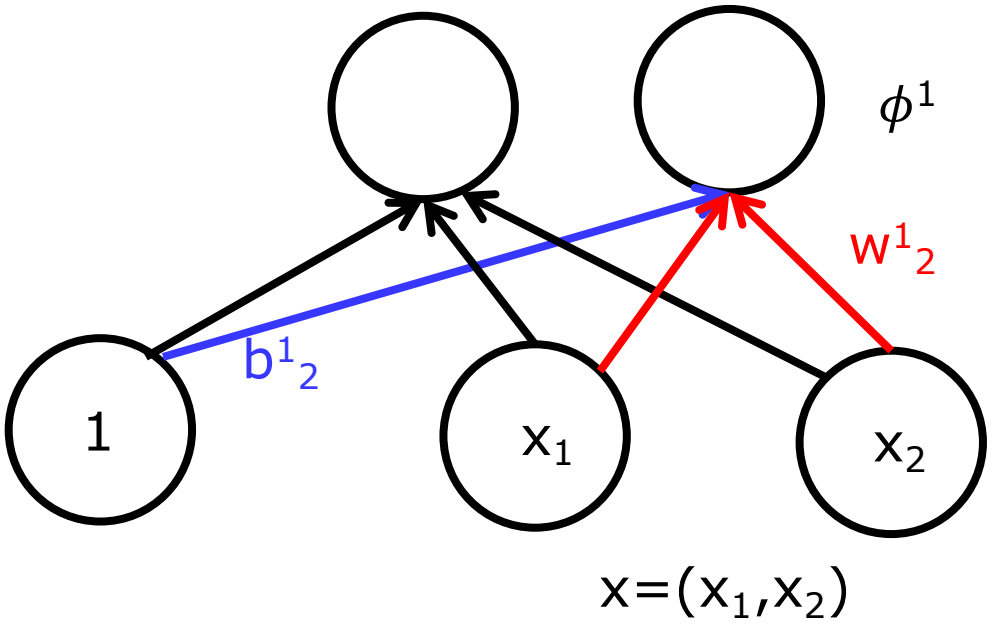
Vectorized Notation

- Fully Connected
  - If x is row vector giving input from non-bias neurons in layer i-1 then xW^i+b^i has j'th entry equal to $xw^i_j+b^i_j$ where w^i_j is j'th column of W^i.



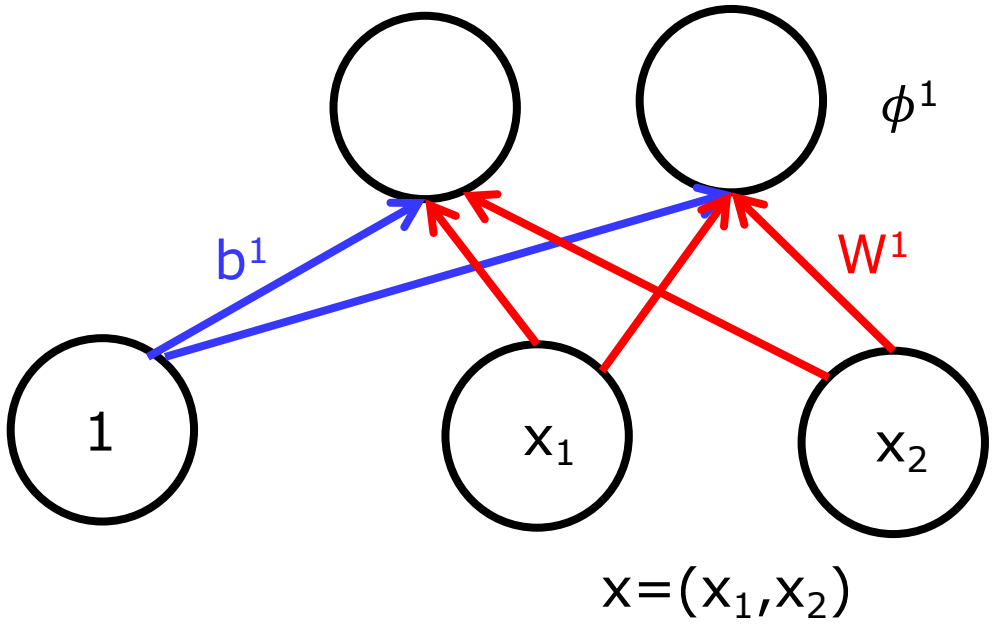
Vectorized Notation

- Fully Connected
  - If x is row vector giving input from non-bias neurons in layer i-1 then xW^i+b^i has j'th entry equal to $xw^i_j+b^i_j$ where w^i_j is j'th column of W^i.



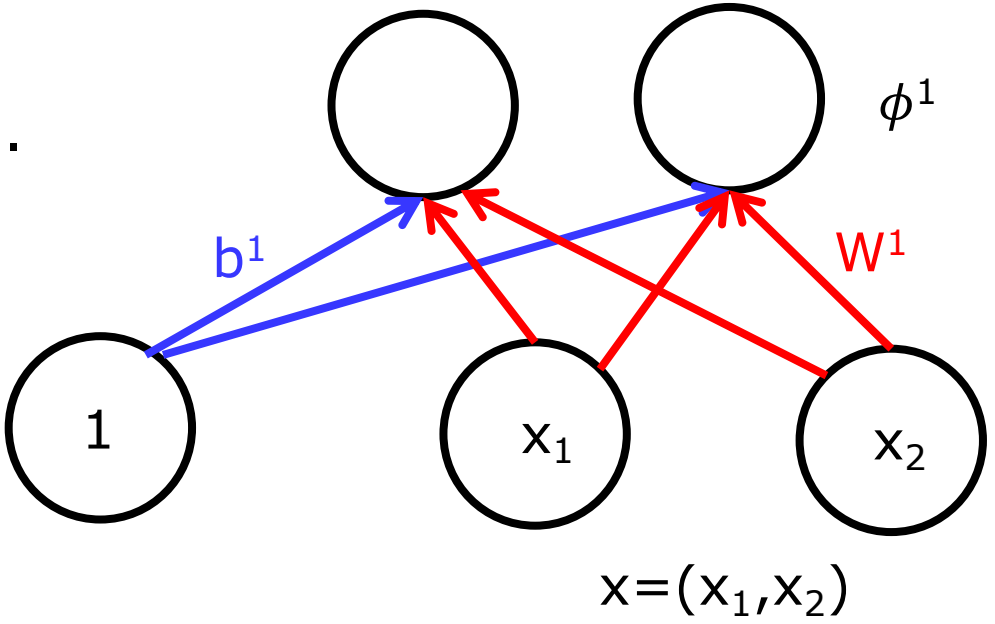
Vectorized Notation

- Fully Connected
  - If x is row vector giving input from non-bias neurons in layer i-1 then xW^i+b^i has j'th entry equal to $xw^i_j+b^i_j$ where w^i_j is j'th column of W^i.
  - xW^i+b^i is output of non-bias neurons in layer i if use Identity activation function.



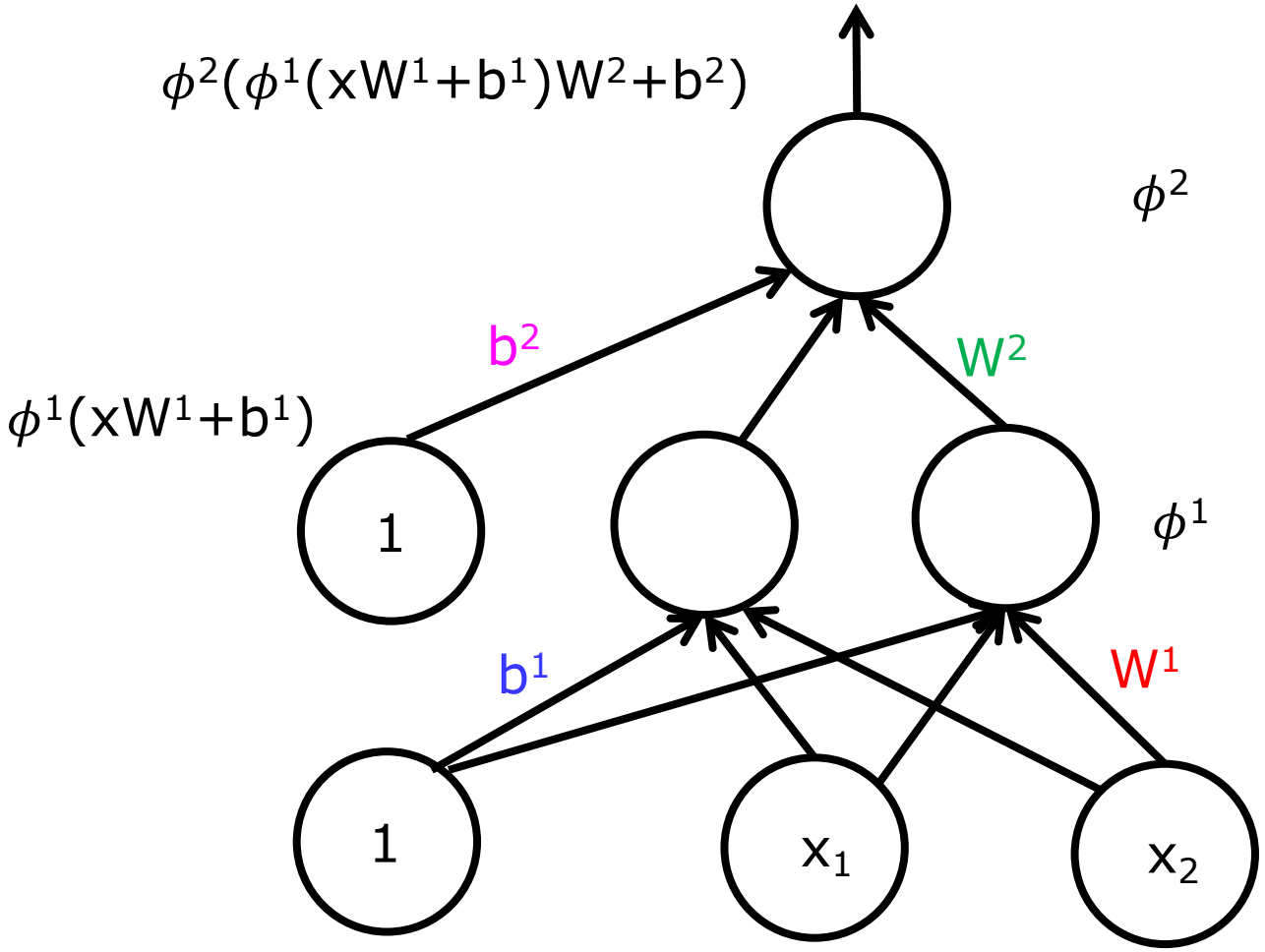
Vectorized Notation

- Fully Connected
  - If x is row vector giving input from non-bias neurons in layer i-1 then xW^i+b^i has j'th entry equal to $xw^i_j+b^i_j$ where w^i_j is j'th column of W^i.
  - $\phi^i(xW^i+b^i)$ is output of non-bias neurons in layer i.
    - ϕ^i is applied entry-wise to a vector.
    - If $v=(v_1,\ldots,v_m)$ then $\phi^i(v)=(\phi^i(v_1),\ldots,\phi^i(v_m))$.



Vectorized Notation

- Fully Connected



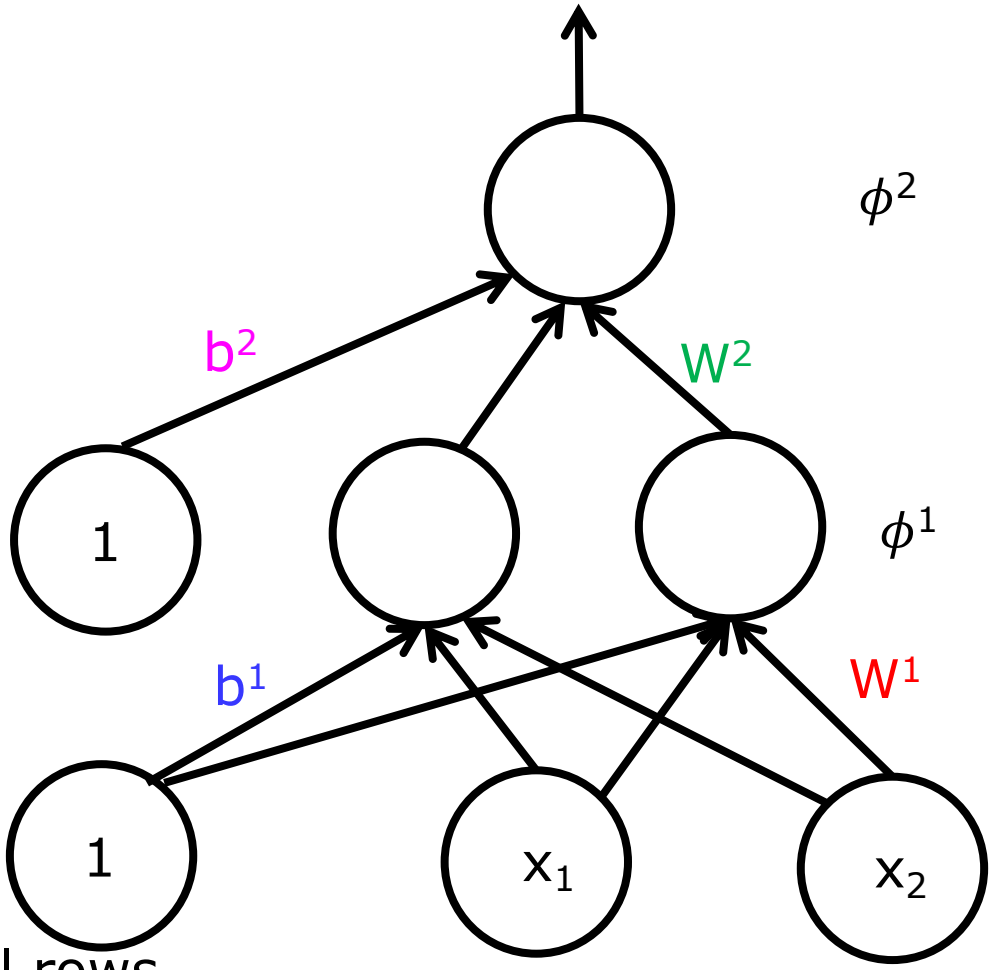
Vectorized Notation

- Fully Connected

$$\phi^k(\phi^{k-1}(\ldots\phi^2(\phi^1(xW^1+b^1)W^2+b^2)\ldots\ )W^k+b^k)$$

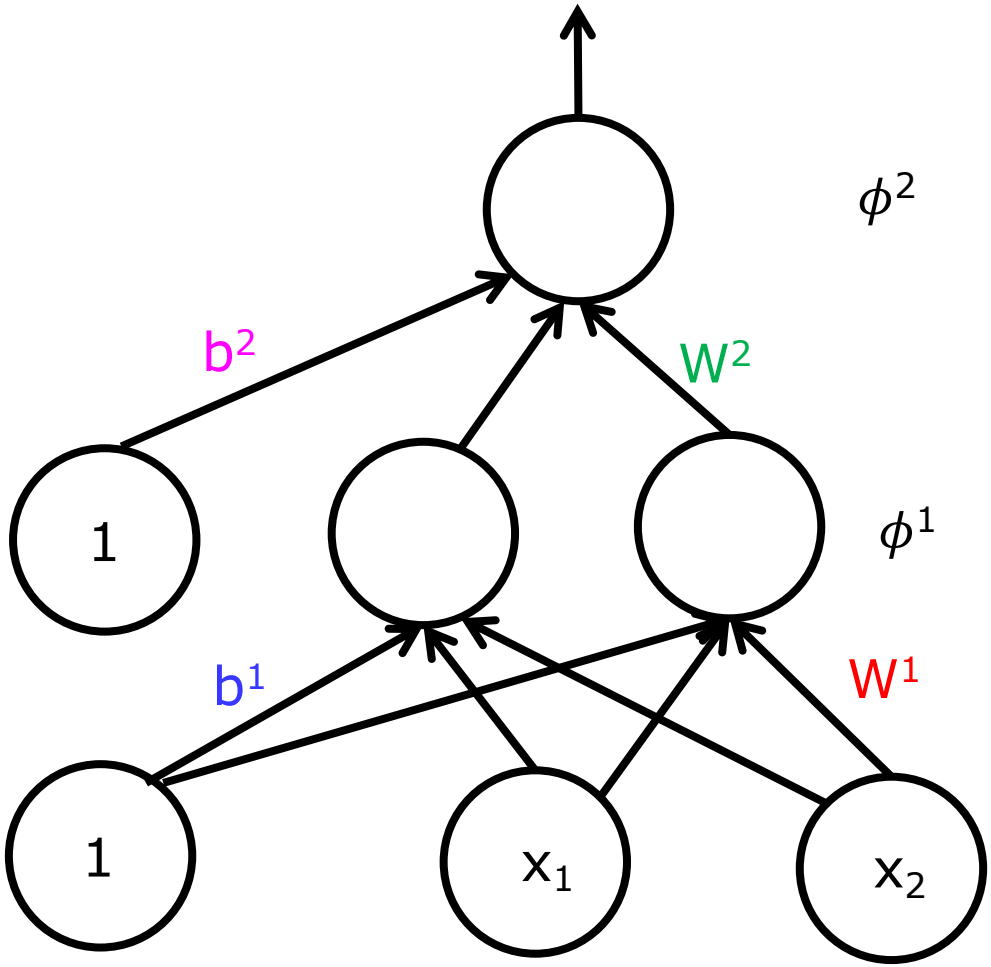
Vectorized Notation

- Fully Connected
  - Batched Evaluation
  - Data matrix:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}$
  - $XW^1 = \begin{pmatrix} x_1 W^1 \\ \vdots \\ x_n W^1 \end{pmatrix}$
  - For nxk matrix A and row vector b with k entries:
    A $\oplus_r$ b is A with b added to all rows.



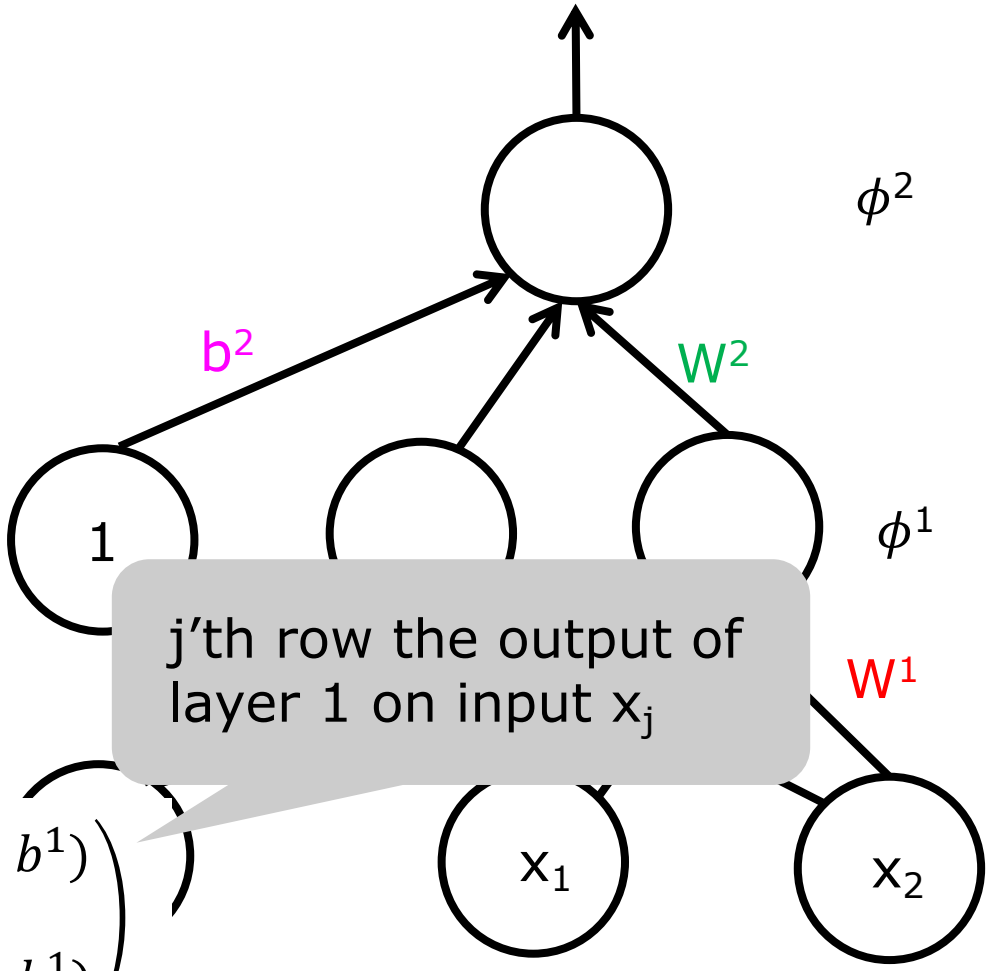
Vectorized Notation

- Fully Connected
  - Batched Evaluation
  - Data matrix:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}$
  - $XW^1 = \begin{pmatrix} x_1 W^1 \\ \vdots \\ x_n W^1 \end{pmatrix}$
  - $XW^1 \oplus_r b^1 = \begin{pmatrix} x_1 W^1 + b^1 \\ \vdots \\ x_n W^1 + b^1 \end{pmatrix}$



Vectorized Notation

- Fully Connected
  - Batched Evaluation
  - Data matrix:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}$
  - $XW^1 \oplus_r b^1 = \begin{pmatrix} x_1W^1 + b^1 \\ \vdots \\ x_nW^1 + b^1 \end{pmatrix}$
  - $\phi^1(XW^1 \oplus_r b^1) = \begin{pmatrix} \phi^1(x_1W^1 + b^1) \\ \vdots \\ \phi^1(x_nW^1 + b^1) \end{pmatrix}$



Vectorized Notation

- Fully Connected
  - Data matrix X with input x_j in j'th row.

$$\phi^k(\phi^{k-1}(\ldots\phi^2(\phi^1(XW^1 \oplus_r b^1)W^2 \oplus_r b^2)\ldots)W^k \oplus_r b^k)$$

Result is a matrix with one row per input x_j and one column per output neuron, containing evaluations.

Remark

- Notation $\oplus_r$
  - For nxk matrix A and row vector b with k entries:
    A $\oplus_r$ b is A with b added to all rows.

```
In [1]: import numpy as np

In [3]: M = np.ones((3,3))

In [4]: print(M)
   [[1. 1. 1.]
    [1. 1. 1.]
    [1. 1. 1.]]

In [5]: v = np.array([1,2,3]).reshape(1, 3)

In [6]: print(v)
   [[1 2 3]]

In [8]: res = M + v
```

```
In [9]: print(res)

[[2. 3. 4.]
 [2. 3. 4.]
 [2. 3. 4.]]
```

Remark

- Notation $\oplus_r$
  - For nxk matrix A and row vector b with k entries:
    A $\oplus_r$ b is A with b added to all rows.

- Using Standard Linear Algebra:
  - Let **1** denote all-1's column vector with n entries.
    A $\oplus_r$ b = A + **1**b

Outer product!

$$\mathbf{1}\mathrm{b} = \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix} (b_1 \quad \cdots \quad b_k) = \begin{pmatrix} b_1 & \cdots & b_k \\ \vdots & \ddots & \vdots \\ b_1 & \cdots & b_k \end{pmatrix}$$